# Introduction to generalized additive models

## day 1 – part 1

Jürgen Claesen

Amsterdam UMC, VU Amsterdam, The Netherlands

March 23, 2026

# Overview

I. Linear Model

II. Scatterplot Smoothing

III. Additive models

# Linear Model

# Introduction

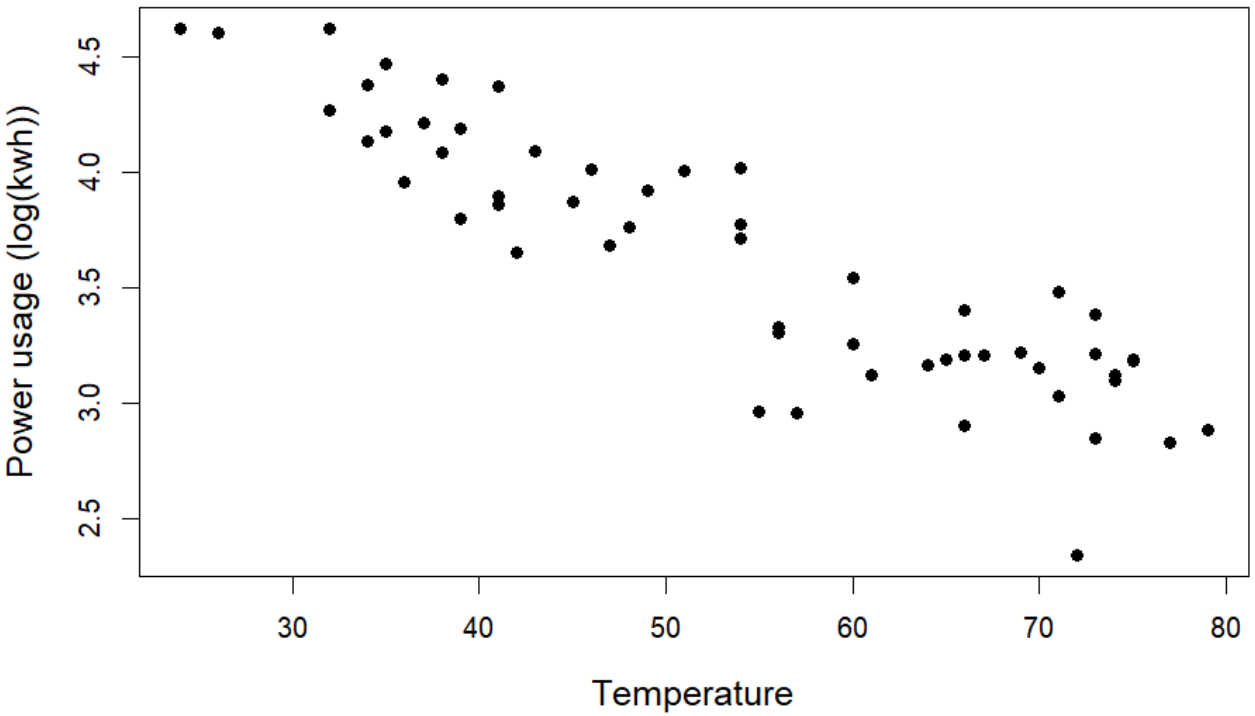



Figure: Scatterplot of Power usage (in log(kwh)) versus Temperature (in Fahrenheit) from an electricity usage study.

Aim of the electricity usage study was to determine the relationship between temperature and power usage.

# Introduction

The scatterplot suggests that a plausible relationship is one of the form:

$$\log \text{kwh}_i = \beta_0 + \beta_1 \times \text{temp}_i + \varepsilon_i$$

with $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$

$\rightarrow$ *simple* linear regression model: one independent variable (*predictor*)

# Linear Model

## Notation

For $p$ predictors, the general form of a linear model is:

$$y_i = \beta_0 + \beta_1 x_{1i} + \cdots + \beta_p x_{1p} + \varepsilon_i$$

Or in matrix notation:

$$\mathrm{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$$

with $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \boldsymbol{I})$

# Linear Model

## Estimation

Point estimates of the linear model parameters, $\boldsymbol{\beta}$, can be obtained by the method of least squares, that is by minimizing:

$$\sum_{i=1}^{n}(y_i - \mathbf{x}_i^T\boldsymbol{\beta})^2 = \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2$$

The least squares estimator of $\boldsymbol{\beta}$ is then:

$$\widehat{\boldsymbol{\beta}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

The influence matrix or hat matrix of a linear regression model:

$$\mathbf{H} = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T$$

and thus

$$\widehat{\mathbf{y}} = \mathbf{H}\mathbf{y}$$

# Linear Model

## Assumptions

Look at the residuals, $\boldsymbol{\varepsilon}$, to check the model assumptions:

$$\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \boldsymbol{I})$$

with $\varepsilon_i = y_i - \widehat{y}_i$

# Linear Model

## Assumptions

i. $E[\varepsilon_i] = 0$

ii. Normally distributed

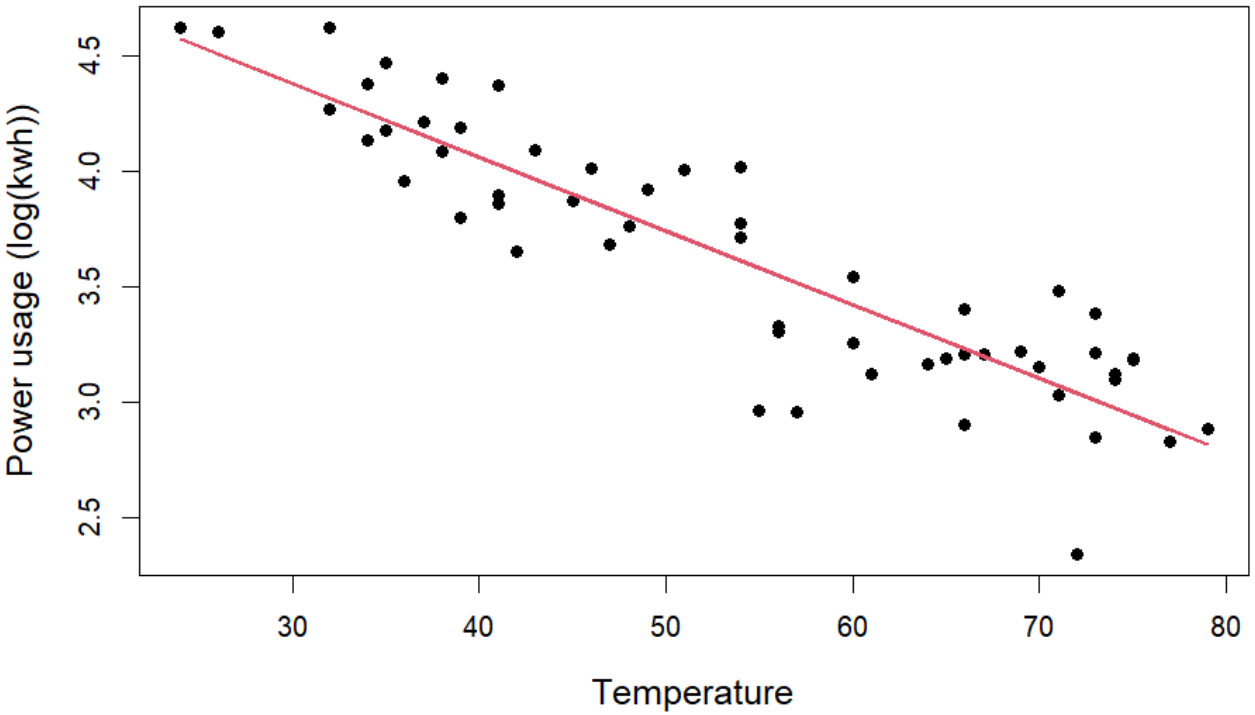
iii. Homoscedastic

iv. Independent & uncorrelated

```
> library(SemiPar)

> data(elec.temp)
> elec.temp$log.usage <- log(elec.temp$usage)

> model1 <- lm(log.usage~temp, data=elec.temp)
> plot(elec.temp$temp, elec.temp$log.usage,
+ pch=16, cex.lab=1.25,
+ xlab="Temperature",
+ ylab="Power usage (log(kwh))")
> lines(elec.temp$temp, fitted(model1), col=2, lwd=2)

> coef(model1)
(Intercept)        temp
 5.33519416 -0.03187381
```

# Linear Models

## Practical




Fitted model:

$$\log \widehat{\text{kwh}}_i = 5.335 - 0.032 \times \text{temp}_i$$

```
> par(mfrow=c(1,2))
> plot(model1, which=c(1:2))

> hist(model1$residuals, main="", xlab="Residuals")
> acf(model1$residuals, main="")
```

# Linear Models

## Practical

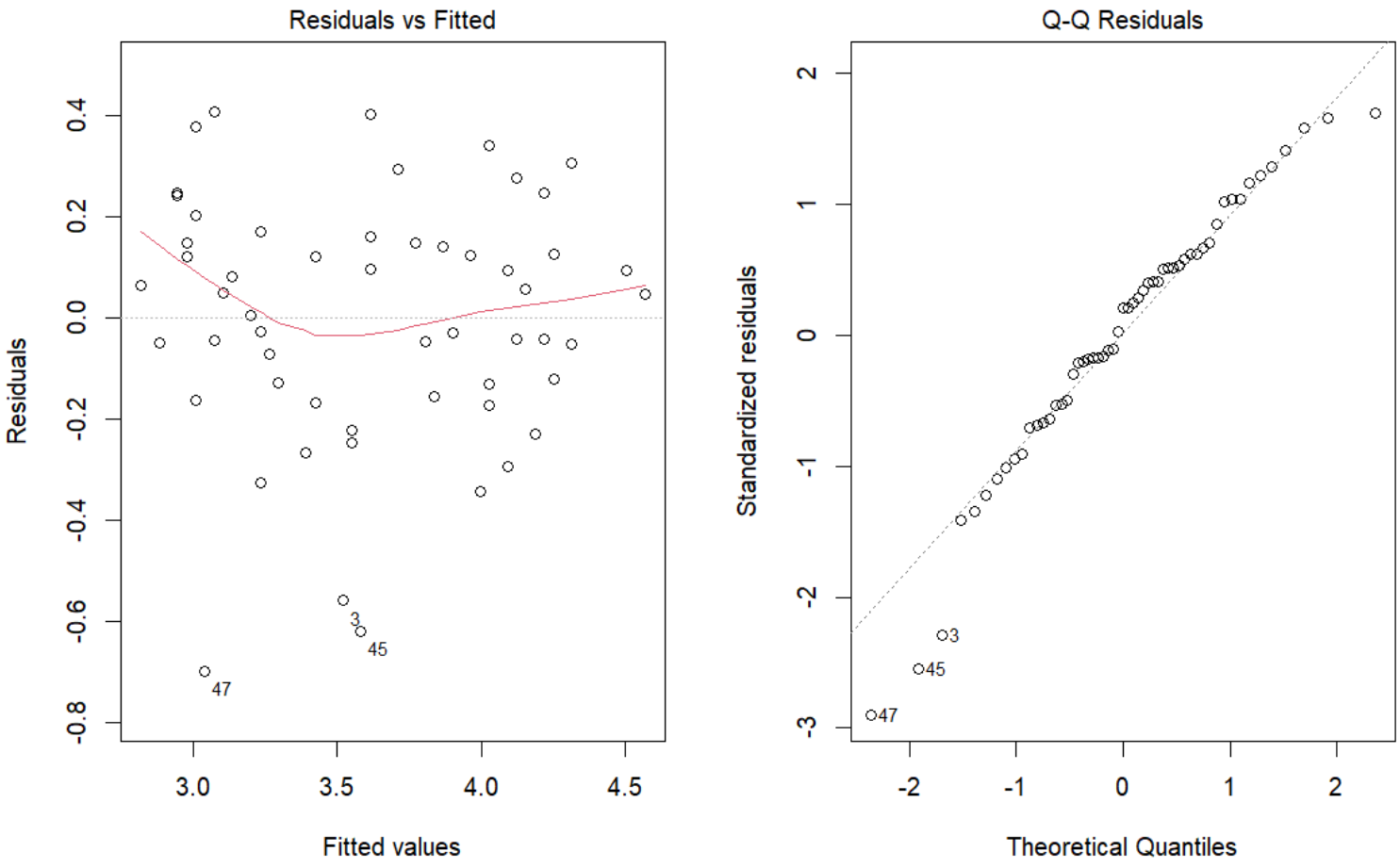



Figure: Residuals vs Fitted (left) and Q-Q plot (right)

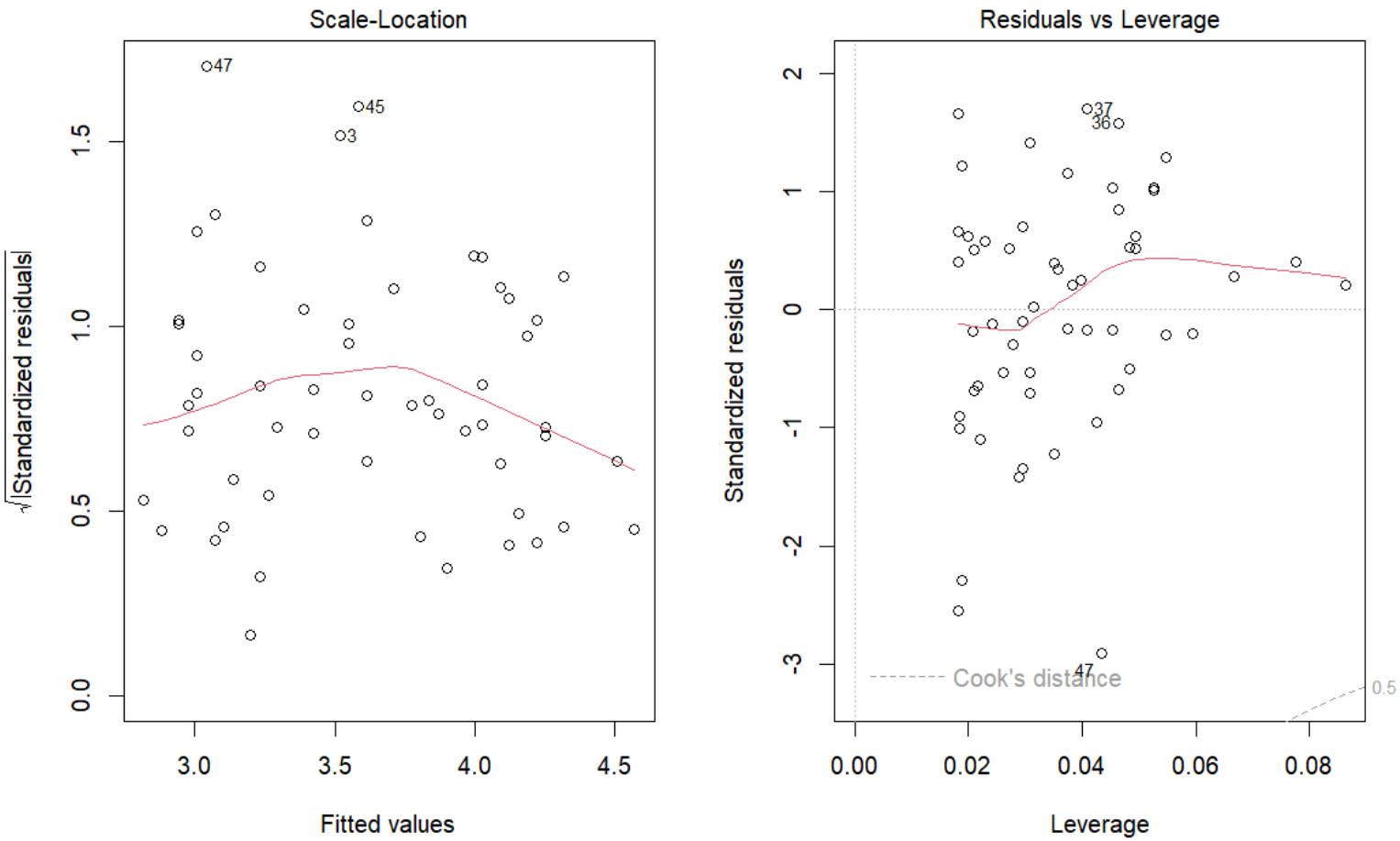



Figure: Standardized Residuals vs Fitted (left) and vs Leverage (right)

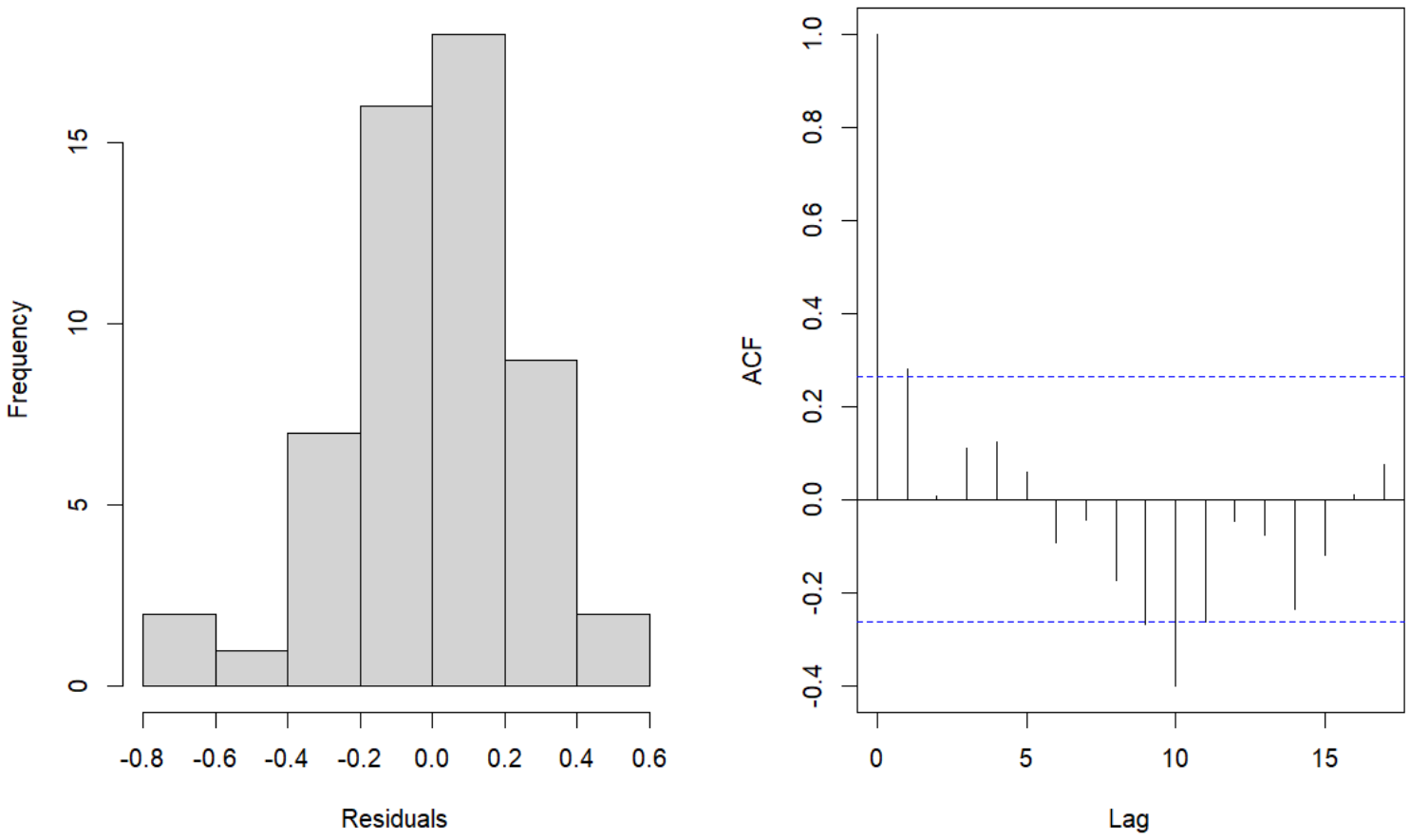



Figure: Histogram of the residuals (left) and autocorrelation plot (right)

# Linear Model

## Assumptions

i. $E(\varepsilon_i) = 0 \rightarrow$ OK

ii. Normally distributed $\rightarrow$ OK

iii. Homoscedastic $\rightarrow$ OK

iv. Independent & uncorrelated $\rightarrow$ not OK

iv. Independent & uncorrelated $\rightarrow$ not OK

Several solutions, such as:

- use additional predictor variables 'temp$_{i-1}$' and 'logkwh$_{i-1}$'
- specify a more general form for Cov($\mathbf{y}$)
- ignore, if correlation between residuals is small

Here, we will ignore the correlation as it is relatively small (and out of convenience)

Recall the least-squares estimator of $\boldsymbol{\beta}$:

$$\widehat{\boldsymbol{\beta}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

As we assume homoscedasticity and independent data, $\text{Cov}(\mathbf{y}) = \sigma^2\mathbf{I}$:

$$\begin{aligned} \text{Cov}(\hat{\boldsymbol{\beta}}) &= \sigma^2(\mathbf{X}^T\mathbf{X})^{-1} \\ \widehat{\text{st.dev}}(\hat{\beta}_i) &= \hat{\sigma}\sqrt{(\mathbf{X}^T\mathbf{X})^{-1})_{ii}} \end{aligned}$$

Assuming normality, the 100(1-$\alpha$)% confidence interval for $\beta_i$ is then:

$$\hat{\beta}_i \pm \widehat{\text{st.dev}}(\hat{\beta}_i)\text{t}(1-\alpha/2, n-p)$$

and the test-statistic for $H_0 : \beta_i = 0$:

$$t(\hat{\beta}_i) = \frac{\hat{\beta}_i}{\widehat{\text{st.dev}}(\hat{\beta}_i)} \overset{H_0}{\sim} \text{t}(n-p)$$

# Linear Model

## Practical

```
> summary(model1)

Call:
lm(formula = log.usage ~ temp, data = elec.temp)

Residuals:
     Min       1Q   Median       3Q      Max
-0.69713 -0.14270  0.04786  0.14938  0.40749

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)  5.335194   0.120312   44.34   <2e-16 ***
temp        -0.031874   0.002149  -14.83   <2e-16 ***
---
```

```
> summary(model1)

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  5.335194   0.120312   44.34   <2e-16 ***
temp        -0.031874   0.002149  -14.83   <2e-16 ***
---

Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘

Residual standard error: 0.2455 on 53 degrees of freedom
Multiple R-squared:  0.8058, Adjusted R-squared:  0.8022
F-statistic: 219.9 on 1 and 53 DF,  p-value: < 2.2e-16
```

$$
\begin{aligned}
E(\hat{y}_i) &= \mathbf{x}_i^T \mathrm{E}(\hat{\boldsymbol{\beta}}) \\
\mathrm{var}(\hat{y}_i) &= \mathbf{x}_i^T \mathrm{Cov}(\hat{\boldsymbol{\beta}})\mathbf{x}_i \\
&= \sigma^2 \mathbf{x}_i^T (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{x}_i
\end{aligned}
$$

The 100(1-$\alpha$)% confidence interval for E($y_i$) is then:

$$\hat{y}_i \pm \widehat{\mathrm{st.dev}}(\hat{y}_i)\mathrm{t}(1-\alpha/2, n-p)$$

$$\begin{aligned} \text{var}(\hat{y}_{pred}) &= \text{var}(\varepsilon) + \text{var}(\hat{y}_i) \\ &= \sigma^2(1 + \mathbf{x}_{new}^T(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{x}_{new}) \end{aligned}$$

The 100(1-$\alpha$)% prediction interval for $y_{new}$ is then:

$$\hat{y}_{pred} \pm \widehat{\text{st.dev}}(\hat{y}_{pred})t(1-\alpha/2, n-p)$$

```
> newD <- data.frame(temp=seq(min(elec.temp$temp),
+ max(elec.temp$temp),by=1))
> resultC <- predict(model1, newD, se.fit=TRUE,
+ interval="confidence")$fit
> resultP <- predict(model1, newD, se.fit=TRUE,
+ interval="prediction")$fit
```

```
> plot(elec.temp$temp, elec.temp$log.usage,
+ pch=16, cex.lab=1.25,
+ xlab="Temperature",
+ ylab="Power usage (log(kwh))")
> lines(elec.temp$temp, fitted(model1), col=2, lwd=2)
> lines(newD$temp, resultC[, "lwr"],
+ col="darkorange", lty=2)
> lines(newD$temp, resultC[, "upr"],
+  col="darkorange", lty=2)
> lines(newD$temp, resultP[, "lwr"],
+ col="purple", lty=3)
> lines(newD$temp, resultP[, "upr"],
+ col="purple", lty=3)
```

# Linear Models

## Practical

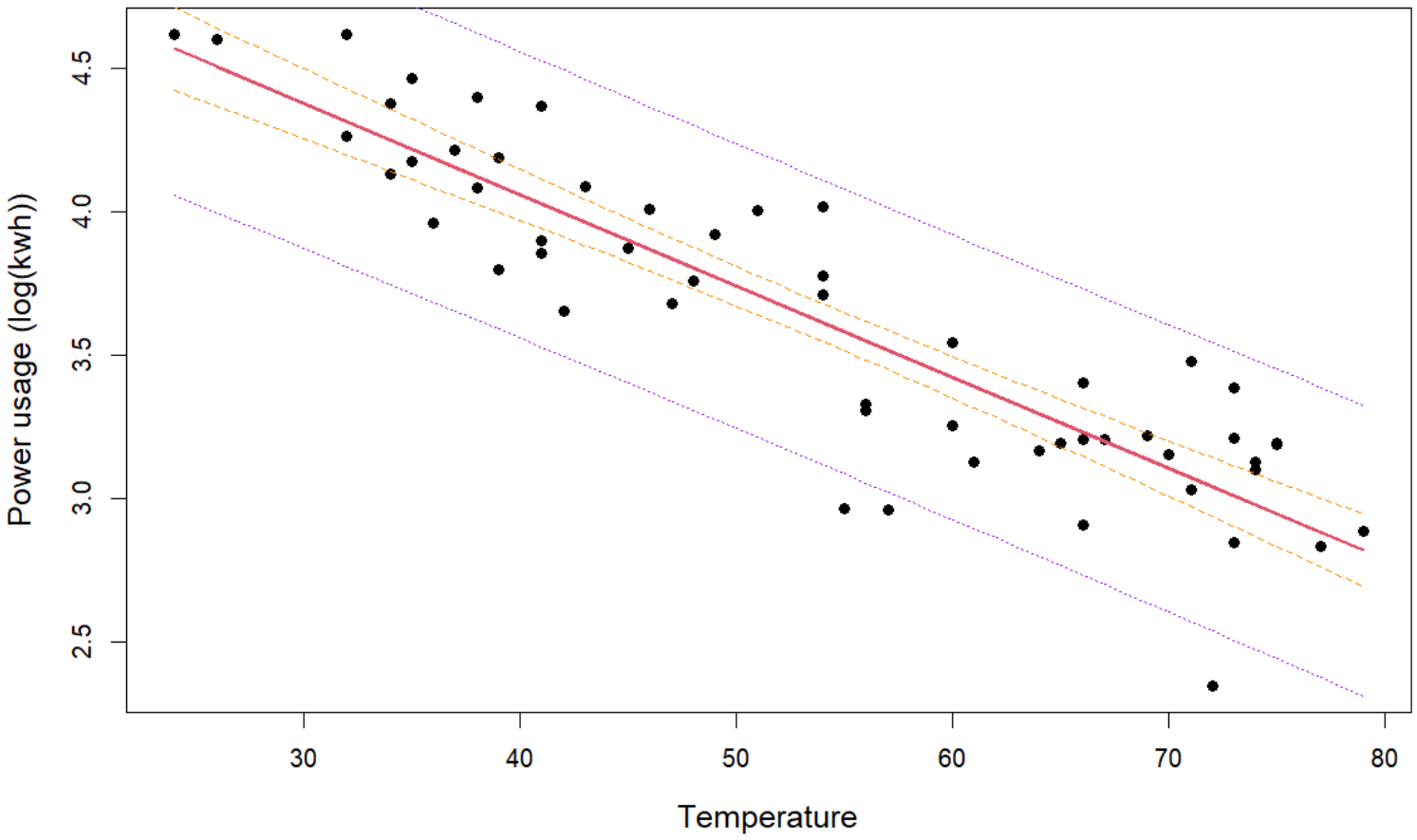



Figure: Fitted model (red) with confidence intervals (orange) and prediction intervals (purple))

# Linear Models

## Exercise 1

# Linear Models

## Non-linear relation

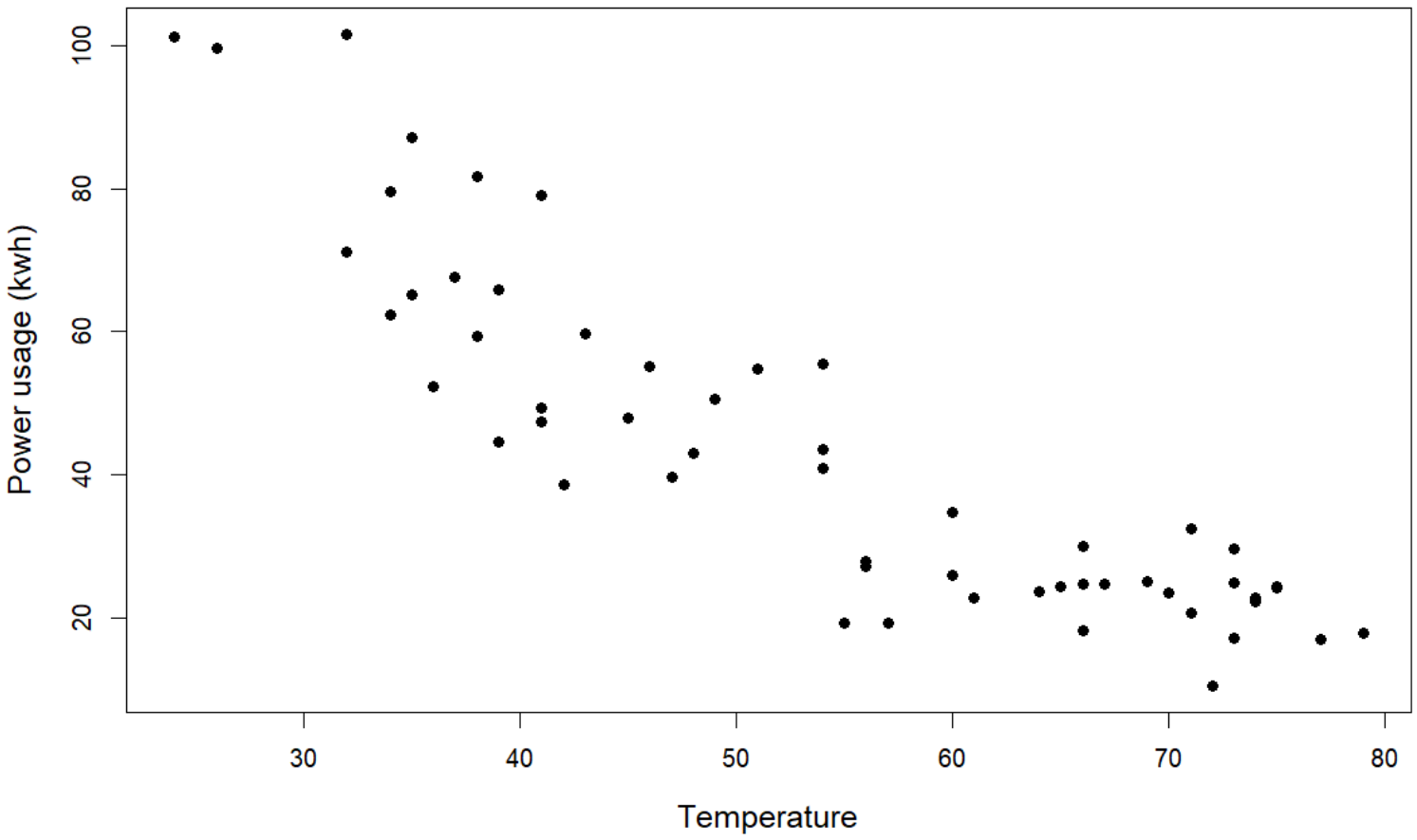



Figure: Scatterplot of Power usage (in kwh) versus Temperature (in Fahrenheit) from an electricity usage study.

Non-linear relation between response (Power usage in (kwh)) and predictor (Temperature)

*Simple* linear regression model $\rightarrow$ inadequate?

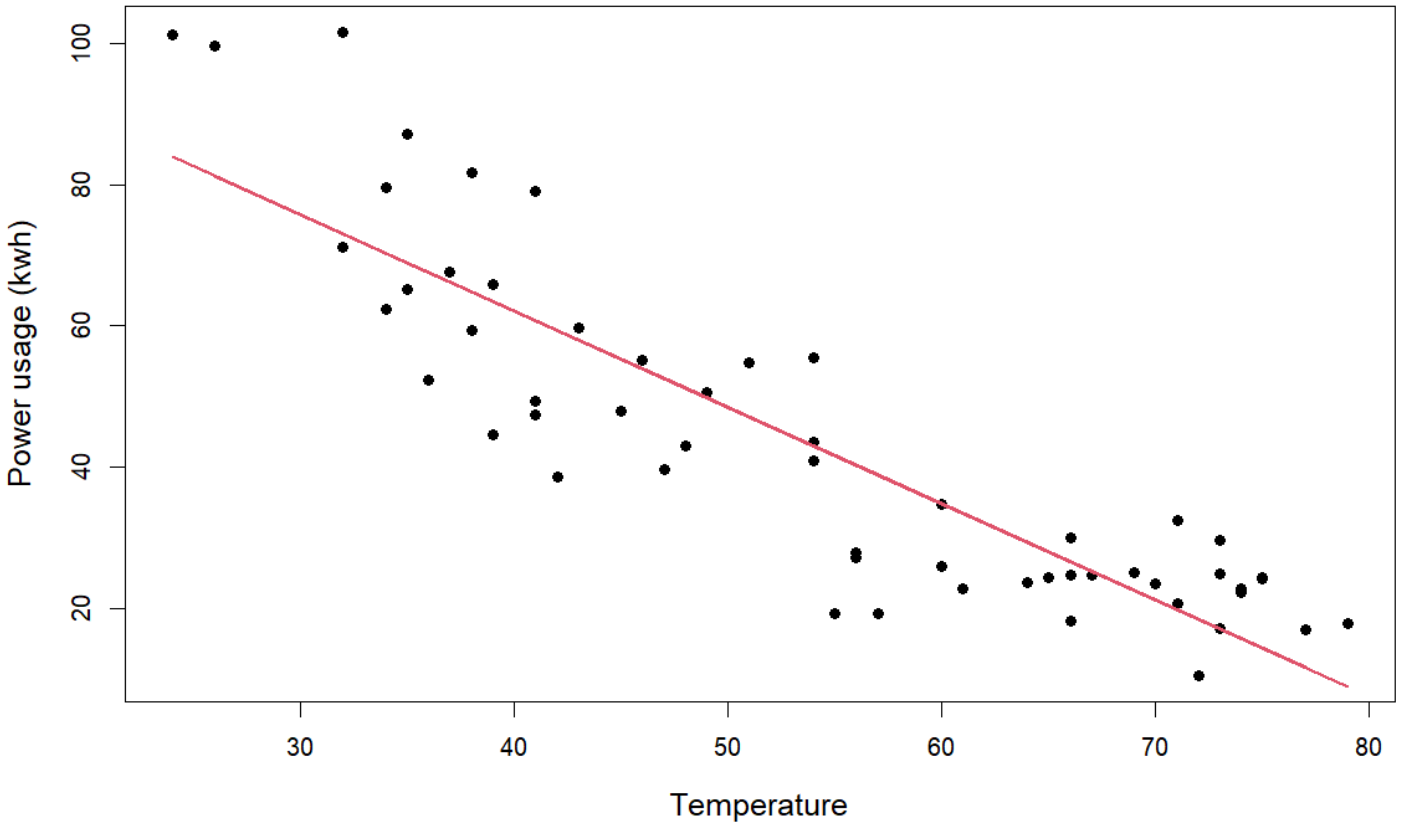
# Linear Model

## Practical

```
> model2 <- lm(usage~temp, data=elec.temp)
> plot(elec.temp$temp, elec.temp$usage,
+ pch=16, cex.lab=1.25,
+ xlab="Temperature",
+ ylab="Power usage (kwh)"
> lines(elec.temp$temp, fitted(model2), col=2, lwd=2)
> coef(model2)
```

# Linear Models

## Non-linear relation




Fitted model:

$$\widehat{\text{kwh}}_i = 116.716 - 1.365 \times \text{temp}_i$$

# Linear Models

## Non-linear relation

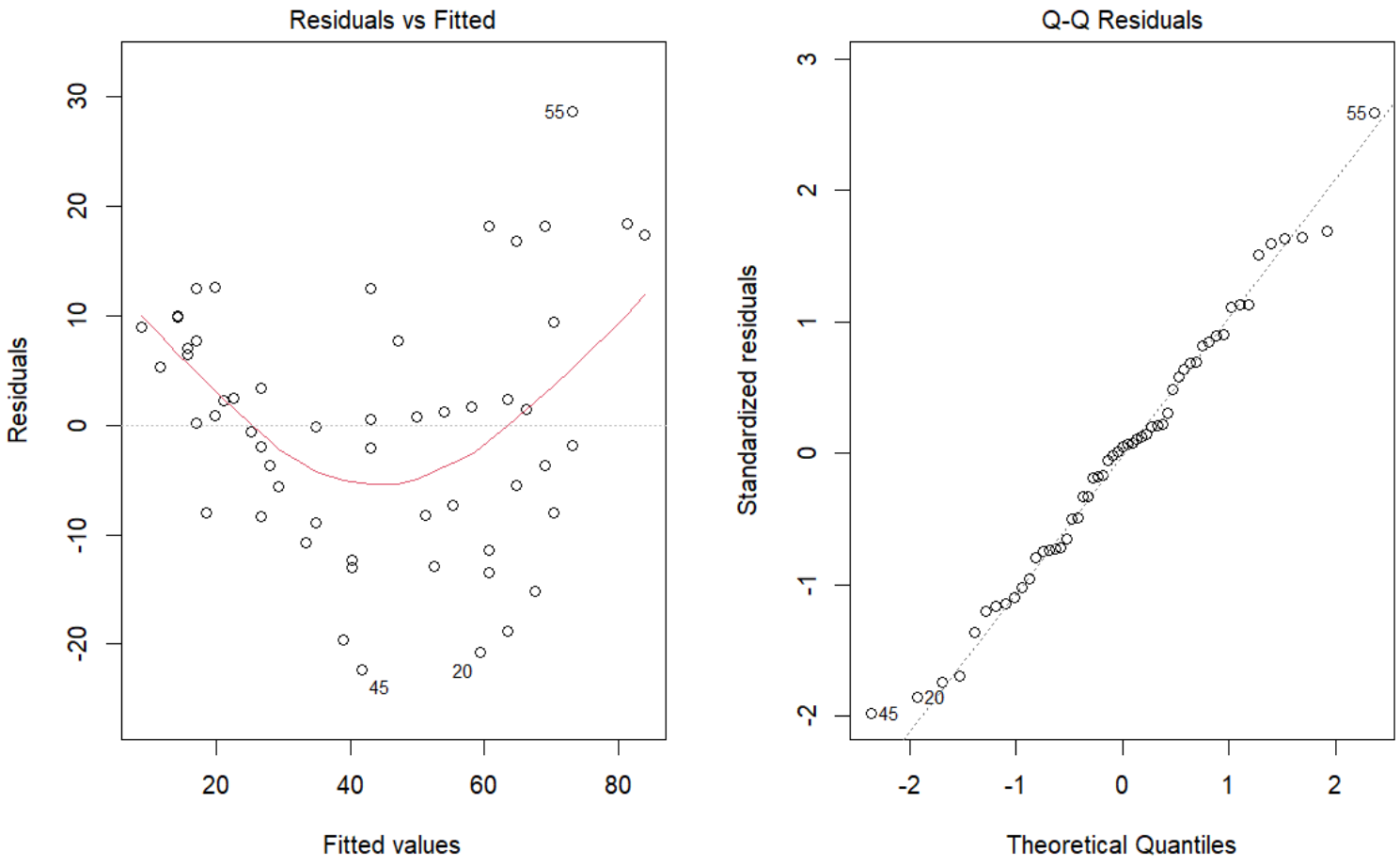



Figure: Residuals vs Fitted (left) and Q-Q plot (right)

Several options to deal with non-linear relation between response and predictor:

i. transformation of one of the variables $\rightarrow$ log(kwh)

ii. non-linear regression model $\rightarrow$ "know" the relation

iii. polynomial regression model

$\rightarrow$ know the relation

$\rightarrow$ unknown/complex relation $\rightarrow$ approximation

The general form of a $k$-th order polynomial regression is:

$$y_i = \beta_0 + \beta_1 x_{1i}^1 + \cdots + \beta_k x_{1i}^k + \varepsilon_i$$

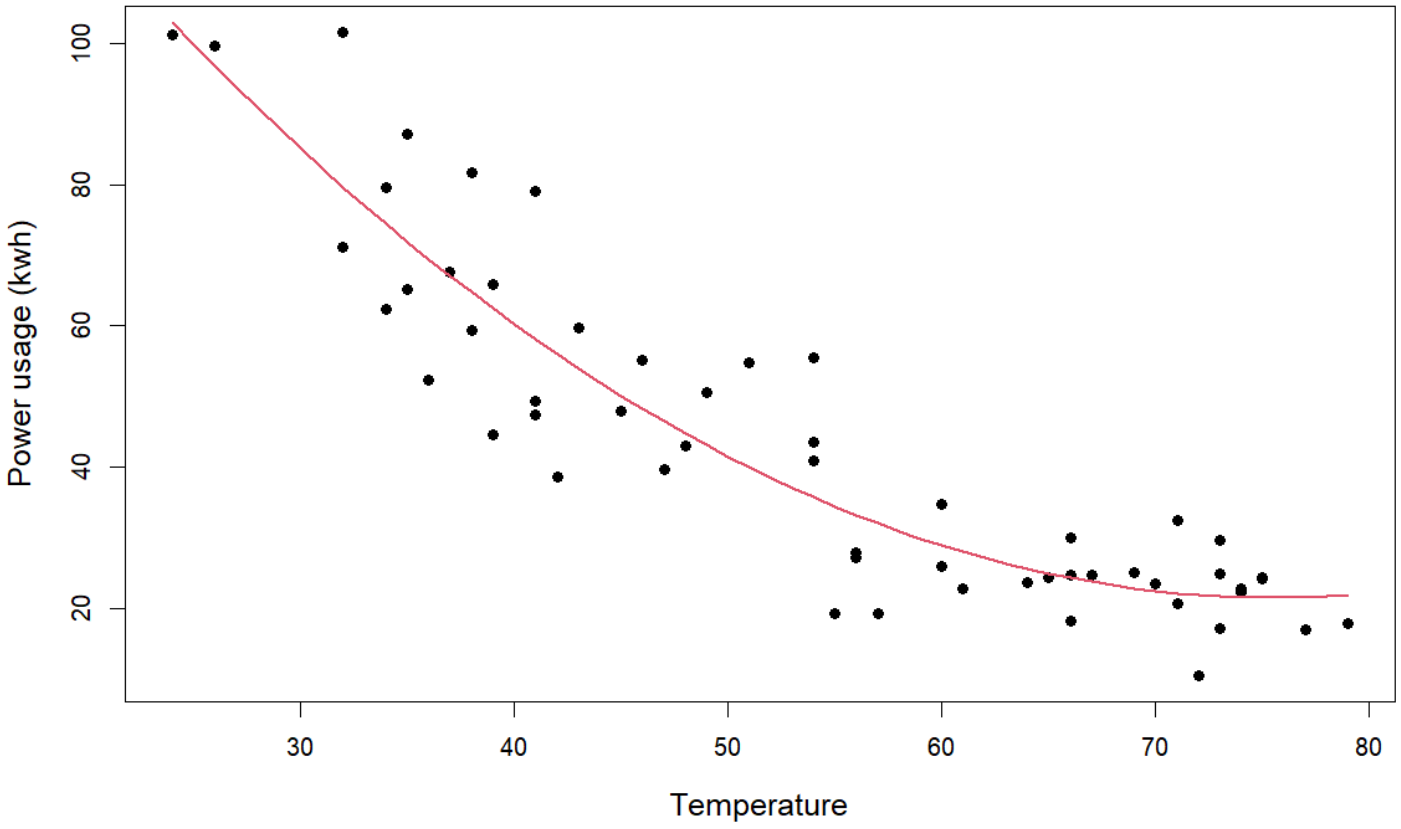
$\rightarrow$ still a linear model $\Rightarrow$ model assumptions, fitting and diagnosis is the same

Issues with multicollinearity are possible $\rightarrow x_i^c = x_i - \bar{x}$

```
> model3 <- lm(usage~temp + I(temp^2), data=elec.temp)
> plot(elec.temp$temp, elec.temp$usage,
+ pch=16, cex.lab=1.25,
+ xlab="Temperature",
+ ylab="Power usage (kwh)"
> lines(elec.temp$temp, fitted(model3), col=2, lwd=2)
> coef(model3)
```

# Linear Models

## Polynomial regression model




Fitted model:

$$\widehat{\text{kwh}}_i = 196.716 - 4.640 \times \text{temp}_i + 0.0307 \times \text{temp}_i^2$$

# Linear Models

## Polynomial regression model

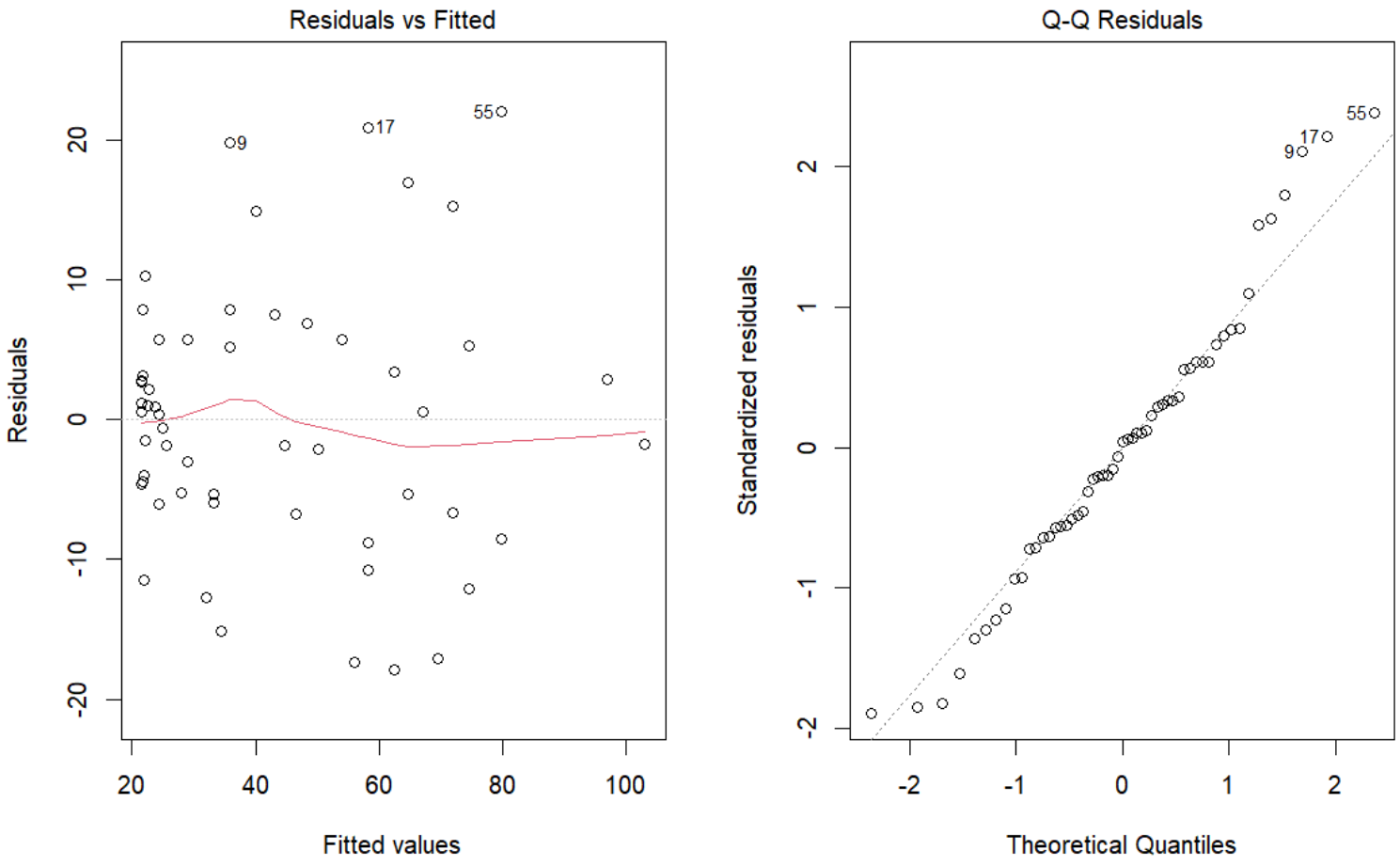



Figure: Residuals vs Fitted (left) and Q-Q plot (right)

# Linear Models

## Polynomial regression model

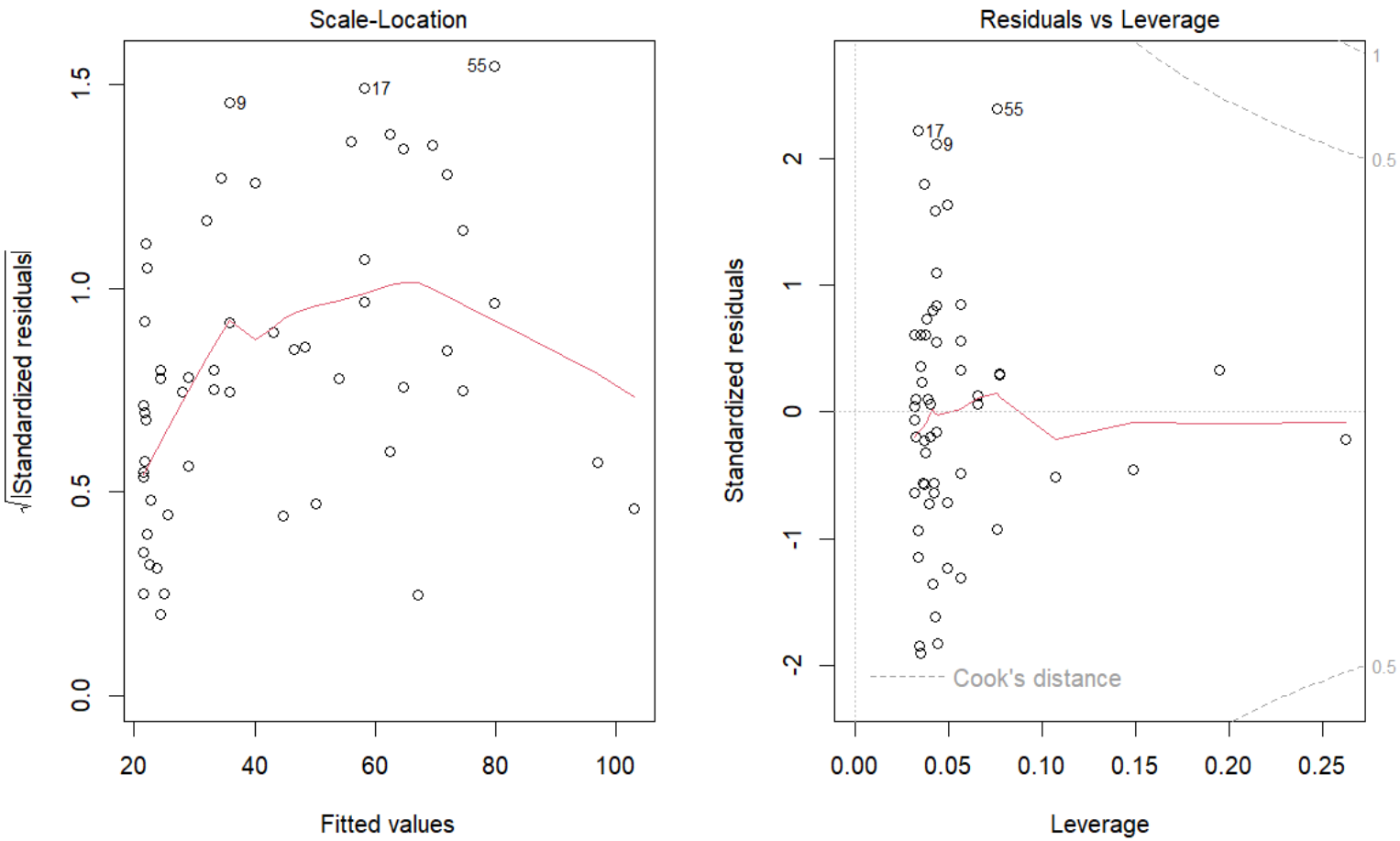



Figure: Standardized Residuals vs Fitted (left) and vs Leverage (right)

Multiple linear regression ($p$ predictors) $\rightarrow 2^p$ possible models

What is the *best* model?

$\rightarrow$ trade-off between *goodness-of-fit* and *parsimony*

Two main approaches:

i. hypothesis testing:
   - include significant parameters ($t$-test, ...)
   - compare models with each other (F-test, LR-test, ...)

ii. predictive power (Residual Sum of Squares)
   - Mallows' $C_p$
   - Aikake's Information Criterium

# Generalized Linear Models

Generalized linear model:

$$\begin{aligned} g(\mu_i) &= \mathbf{X}_i \boldsymbol{\beta} \\ \mathrm{E}(y_i) &\equiv \mu_i \end{aligned}$$

- response $y_i$: distribution from family of exponential distributions: Binomial, Poisson, Gamma, Normal, ....
- some non-linearity in the model structure due to the link function $g(.)$

Many of the general ideas and concepts of linear modelling carry over, with a little modification, to generalized linear modelling (link function and distribution)

# Scatterplot Smoothing

# Introduction

Several options to deal with non-linear relation between response and predictor:

i. transformation of one of the variables $\rightarrow$ log(kwh)

ii. non-linear regression model $\rightarrow$ "know" the relation

iii. polynomial regression model
   - $\rightarrow$ known relation
   - $\rightarrow$ unknown relation $\rightarrow$ approximation $\Rightarrow$ scatterplot smoothing

# Scatterplot smoothing

## Introduction

For a single covariate $x$:

$$y_i = f(x_i) + \varepsilon_i$$

with $f$ a smooth function and $\varepsilon$ independent random variables

Prefer that the form of $f$ (*basis*) is such that the model remains a linear model

A *basis* is set of basis functions, which are treated as completely known

$$f(x) = \sum_{j=1}^{k} b_j(x)\beta_j$$

with $b_j$ the $j^{th}$ basis function

# Scatterplot smoothing

## Polynomial bases

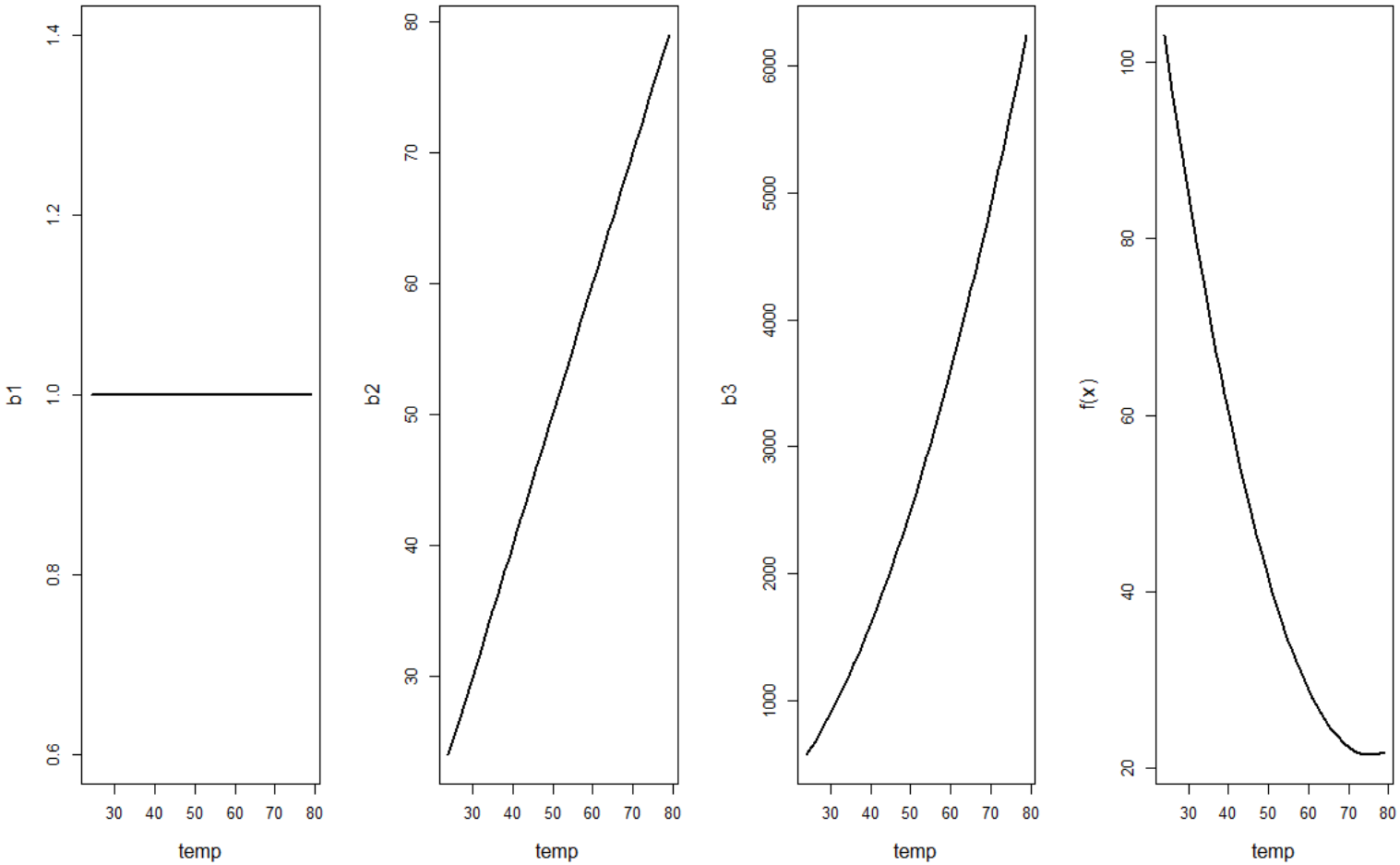
The $2^{nd}$ order polynomial model from our example:

$$y_i = \beta_1 + \beta_2 x_{1i}^1 + \beta_3 x_{1i}^2 + \varepsilon_i$$

has as basis: $b_1(x) = 1$, $b_2(x) = x$, $b_3(x) = x^2$

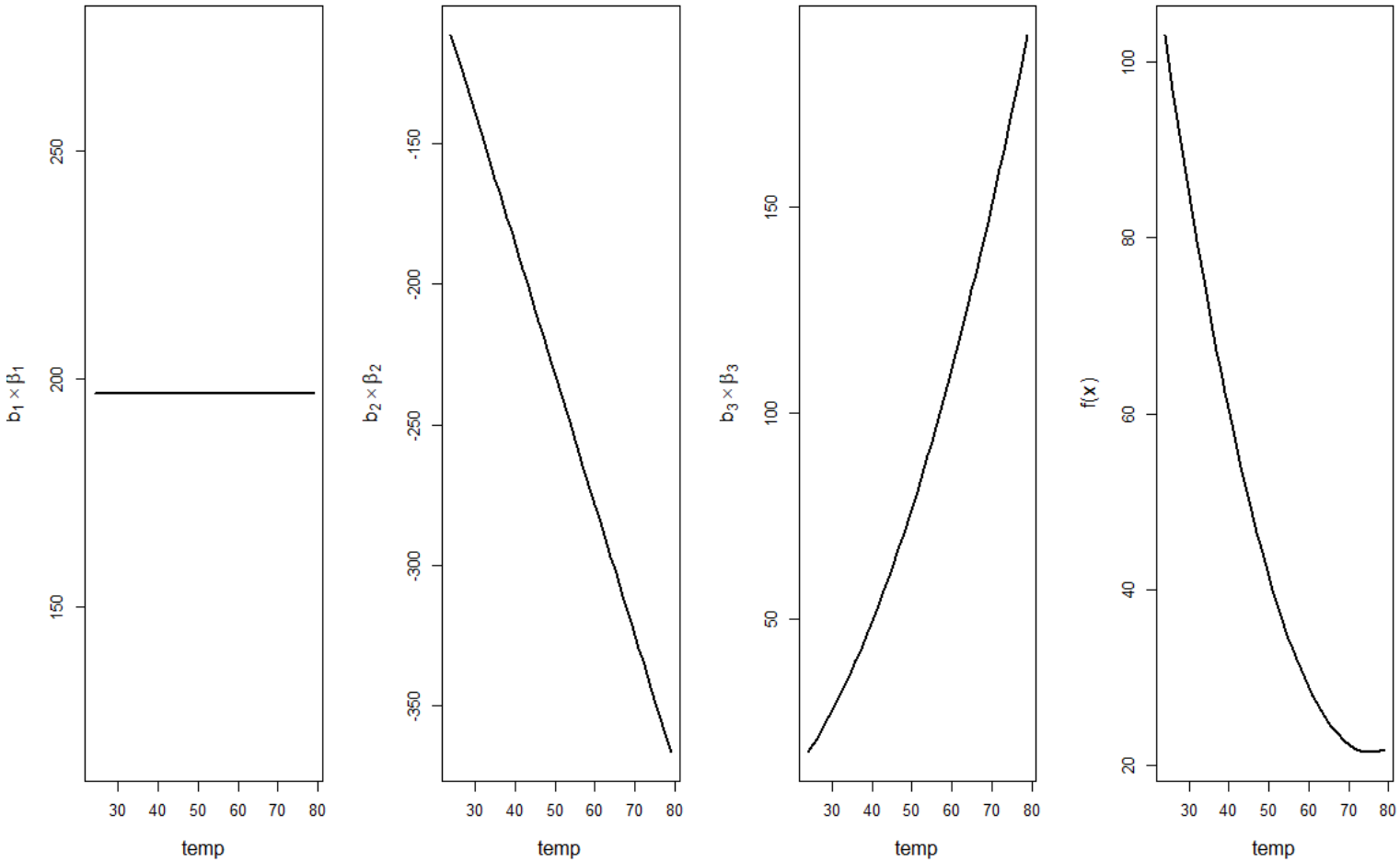
# Scatterplot smoothing

## Polynomial bases



# Scatterplot smoothing

## Polynomial bases



Polynomial bases:

- useful when interest focuses on properties of $f$ in the vicinity of a single specified point.
- problematic when interest focuses on the whole domain of $f$

# Scatterplot smoothing

## Polynomial bases

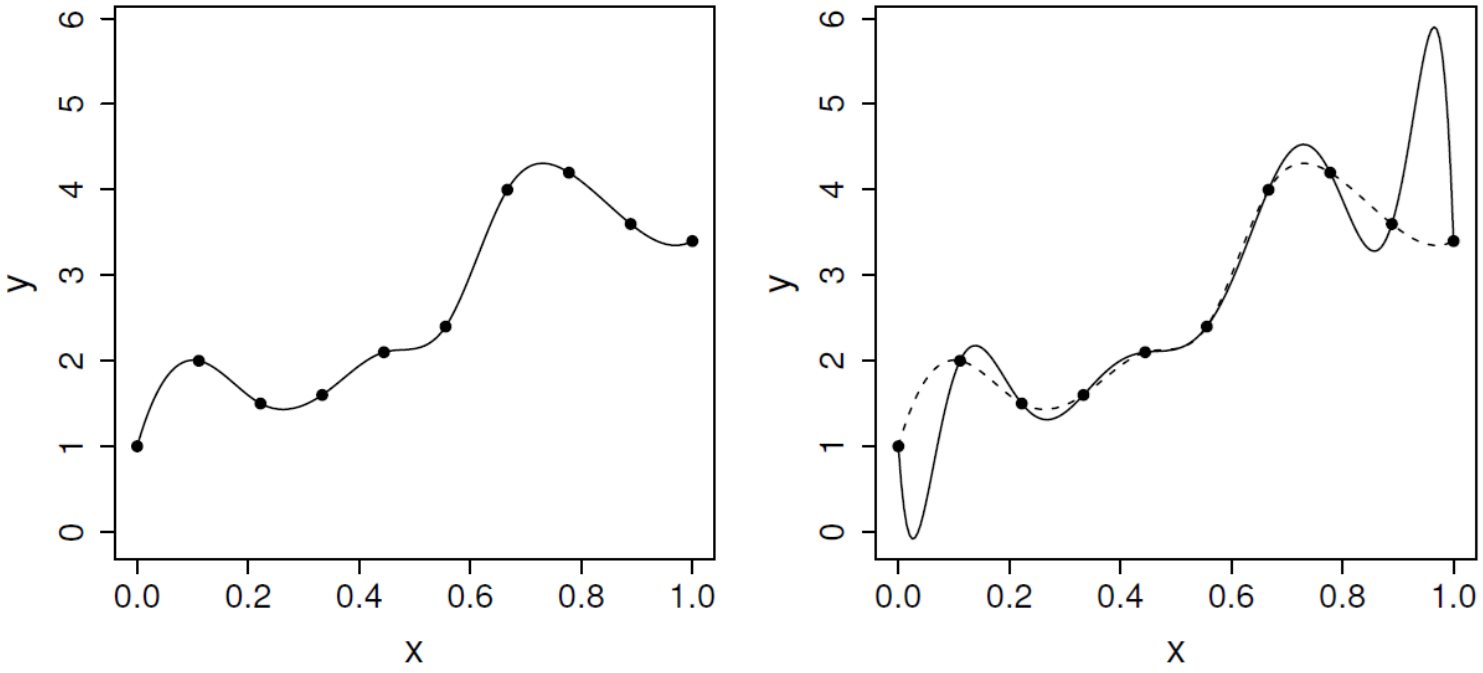



Figure: The left panel shows a smooth function sampled at the points shown as black dots. The right panel shows an attempt to reconstruct the function (dashed curve) by polynomial interpolation (black curve) of the black dots.[2]

[2] Taken from Wood, 2017

# Scatterplot smoothing

## Piecewise Linear basis

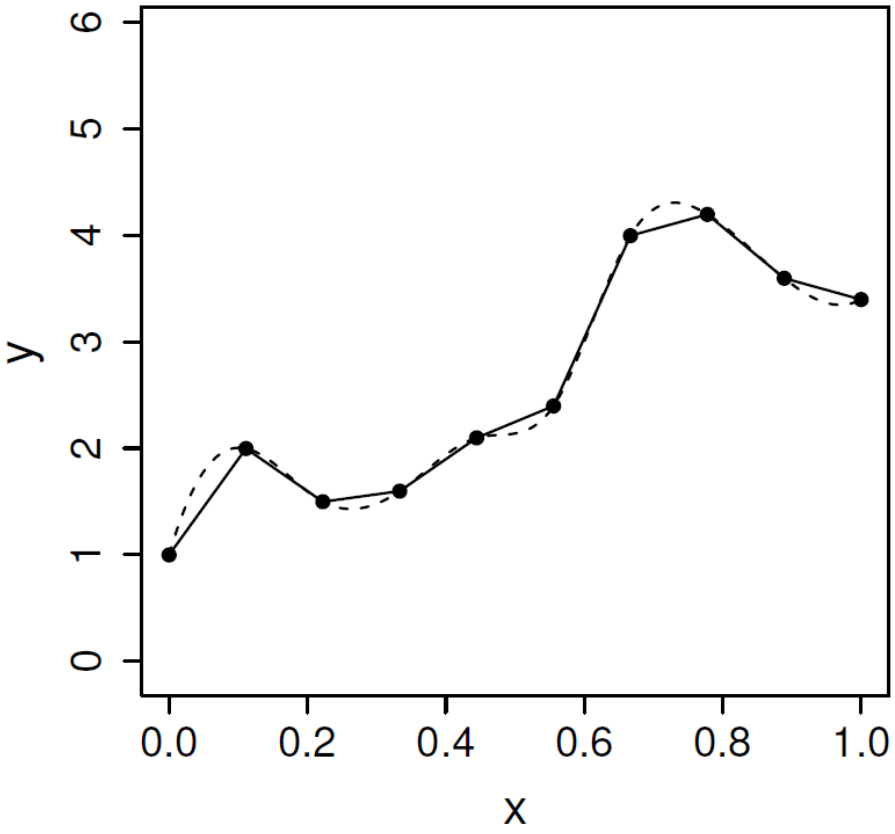



Figure: Reconstruction of the function (dashed curve) by a piecewise linear basis (black curve) of the black dots.[4]

[4] Taken from Wood, 2017

# Scatterplot smoothing

## Piecewise Linear basis

An alternative basis is the piecewise linear basis:

- good at approximating unknown functions
- performs well for interpolating exact observations of a function

# Scatterplot smoothing

## Piecewise Linear basis

Locations at which linear pieces "connect" are *knots* ($x_j^*$)

An example of a piecewise linear basis functions are *tent functions*.

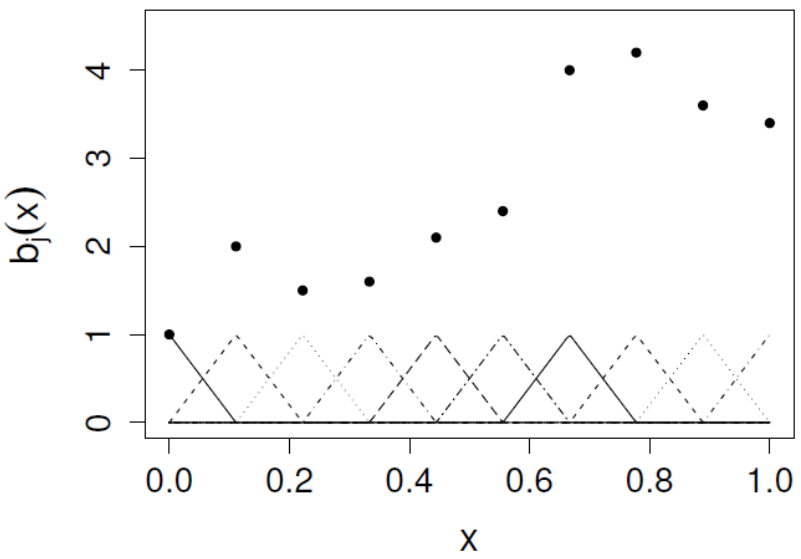



Figure: Tent function basis for interpolating the data (black dots)[5].

[5] Taken from Wood, 2017

# Scatterplot smoothing

## Piecewise Linear basis – tent functions

Suppose that $x_j^* > x_{j-1}^*$ and

$$b_1(x) = \begin{cases} (x_2^* - x)/(x_2^* - x_1^*) \text{ if } x < x_2^* \\ 0 \text{ otherwise} \end{cases}$$

and

$$b_k(x) = \begin{cases} (x - x_{k-1}^*)/(x_k^* - x_{k-1}^*) \text{ if } x > x_{k-1}^* \\ 0 \text{ otherwise} \end{cases}$$

then for $j = 2, \ldots, k-1$:

$$b_j(x) = \begin{cases} (x - x_{j-1}^*)/(x_j^* - x_{j-1}^*) \text{ if } x_{j-1}^* < x \leq x_j^* \\ (x_{j+1}^* - x)/(x_{j+1}^* - x_j^*) \text{ if } x_j^* < x < x_{j+1}^* \\ 0 \text{ otherwise} \end{cases}$$

# Scatterplot smoothing

## Piecewise Linear basis – tent functions

$b_j(x)$ is zero everywhere, except over the interval between the knots immediately to either side of $x_j^*$.

$b_j(x)$ increases linearly from 0 at $x_{j-1}^*$ to 1 at $x_j^*$, and then decreases linearly to 0 at $x_{j+1}^*$

We will use here the LIDAR dataset (Holst et al., 1996). LIDAR is commonly used to monitor air pollutants. The dataset consists of two variables: *range* and *logratio*. The variable *range* is the distance traveled before the light is reflected back to its source. The response variable (*logratio*) is the logarithm of the ratio of received light from two laser sources. One source had a frequency equal to the resonance frequency of the compound of interest, which was mercury in this study. The other source had a frequency off this resonance frequency.

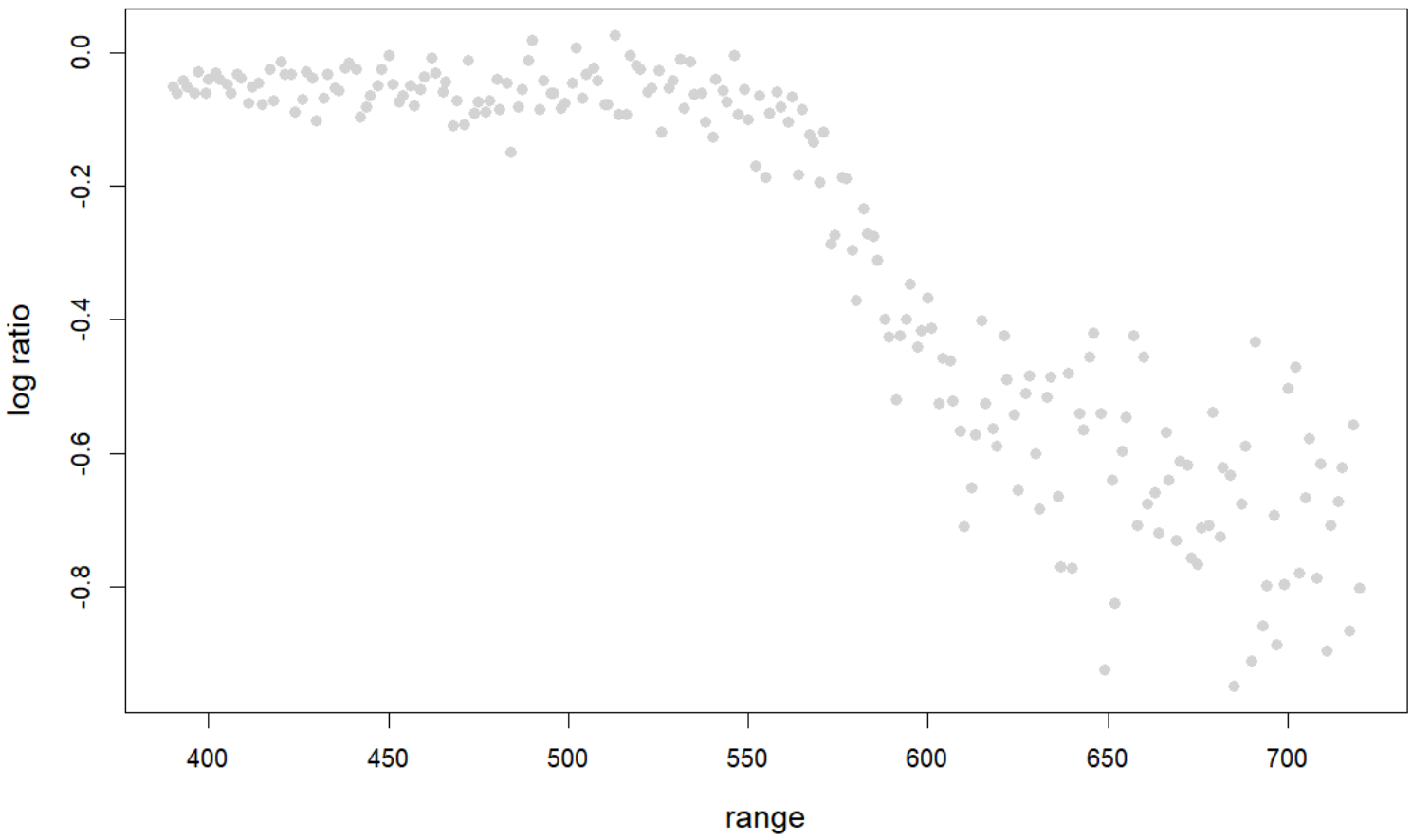
# Scatterplot smoothing

## Piecewise Linear basis – practical

```
> library(SemiPar)
> data(lidar)
> plot(x=lidar$range,lidar$logratio,
+ pch=16, cex.lab=1.25, col="lightgray",
+ xlab="range", ylab="log ratio")
```

# Scatterplot smoothing

## Piecewise Linear basis – practical



# Scatterplot smoothing

## Piecewise Linear basis – practical

```
> tf <- function(x,xj,j) {
+   ## generate jth tent function from set
+   ##defined by knots xj
+   dj <- xj*0;dj[j] <- 1
+   approx(xj,dj,x)$y
+ }

> tf.X <- function(x,xj) {
+   ## tent function basis matrix given data x
+   ## and knot sequence xj
+   nk <- length(xj); n <- length(x)
+   X <- matrix(NA,n,nk)
+   for (j in 1:nk) X[,j] <- tf(x,xj,j)
+   X
+ }
```

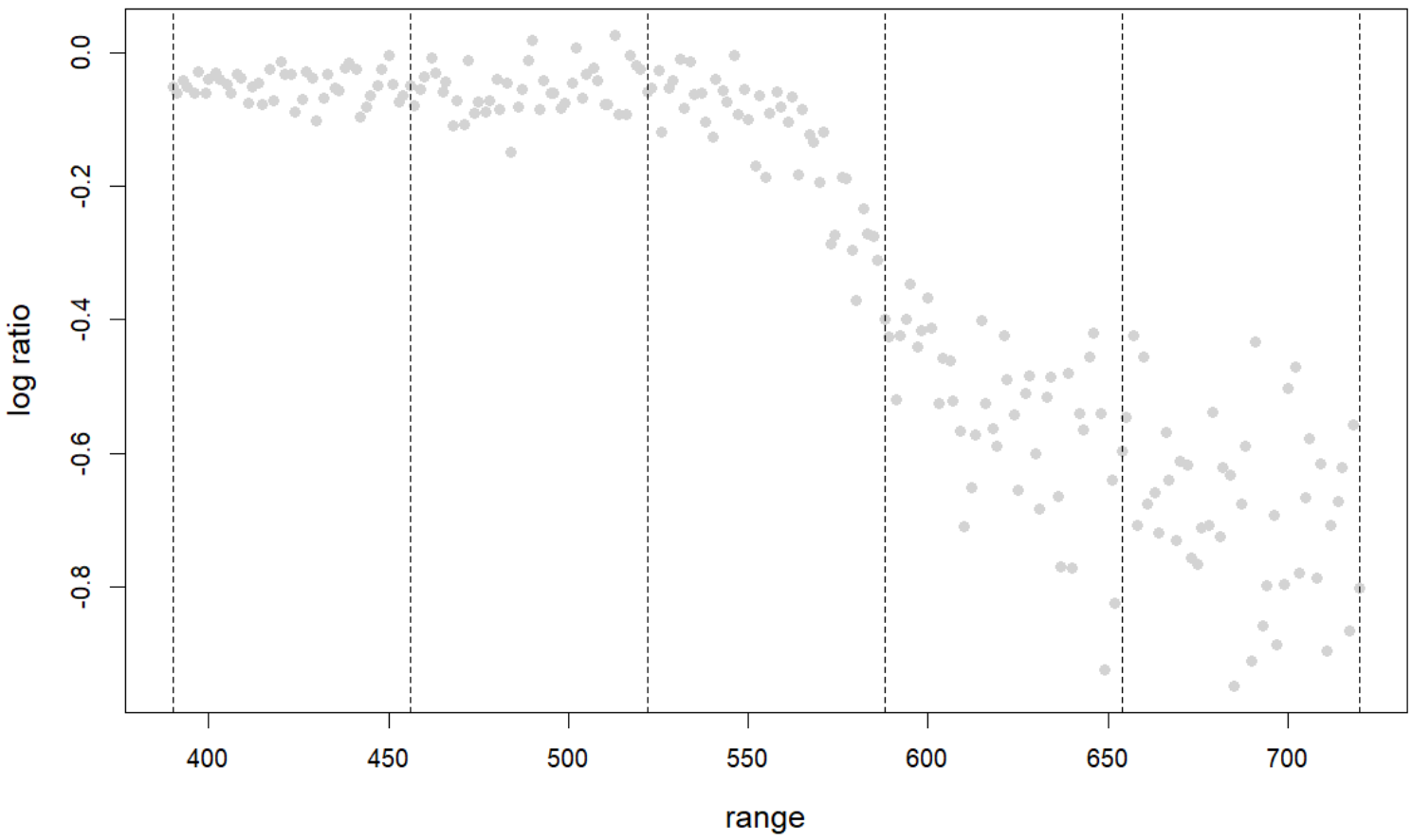
# Scatterplot smoothing

## Piecewise Linear basis – practical

```
## generate knots
> sj <- seq(min(lidar$range),max(lidar$range),length=6)
> X <- tf.X(lidar$range,sj) ## get model matrix
> b <- lm(lidar$logratio ~ X - 1) ## fit model
```

# Scatterplot smoothing

## Piecewise Linear basis – practical



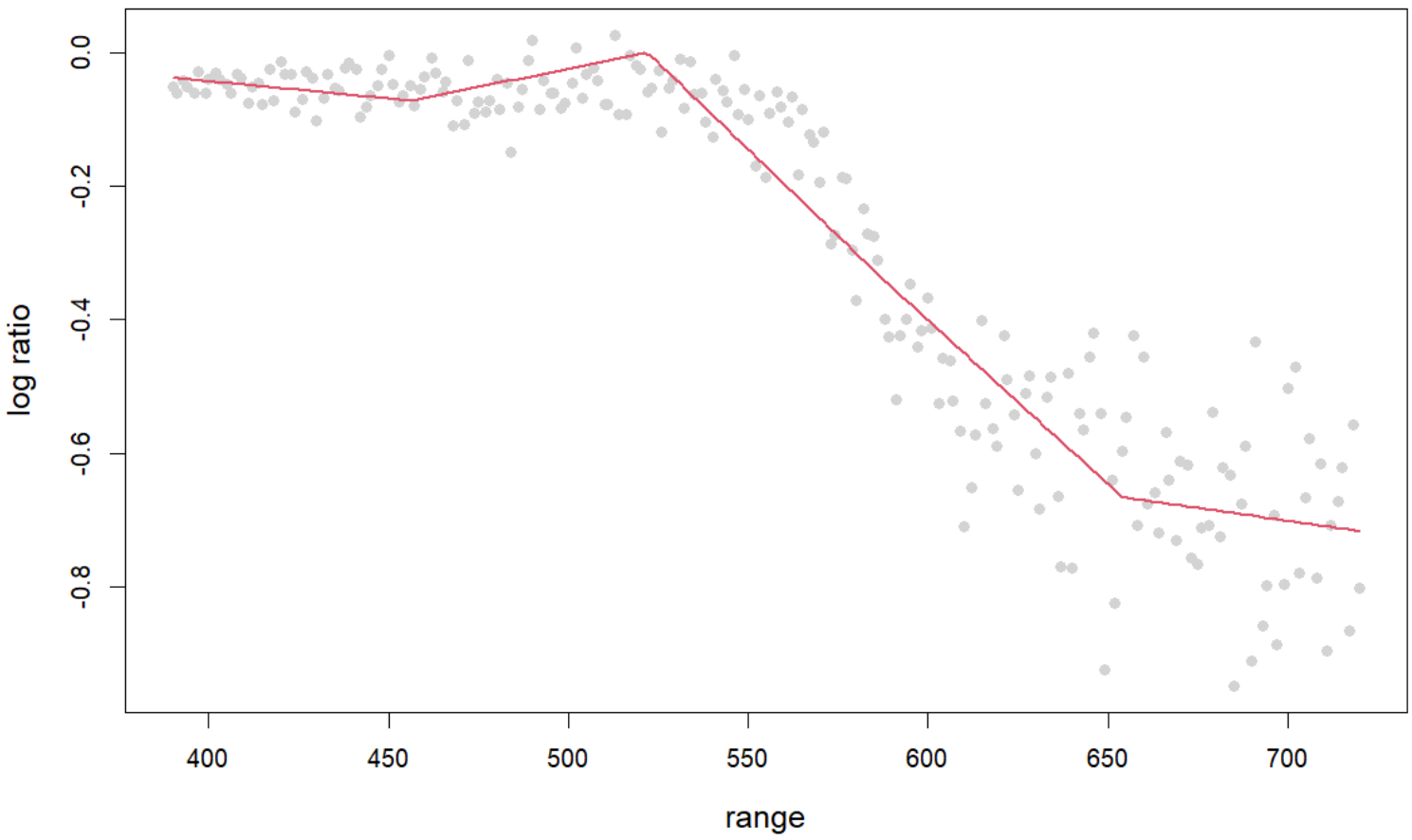
# Scatterplot smoothing

## Piecewise Linear basis – practical

```
## prediction data
> s <- seq(min(lidar$range), max(lidar$range), length=200)
> Xp <- tf.X(s,sj) ## prediction matrix
## overlay estimated f
> lines(s,Xp %*% coef(b), lwd=2, col=2)
```

# Scatterplot smoothing

## Piecewise Linear basis – practical



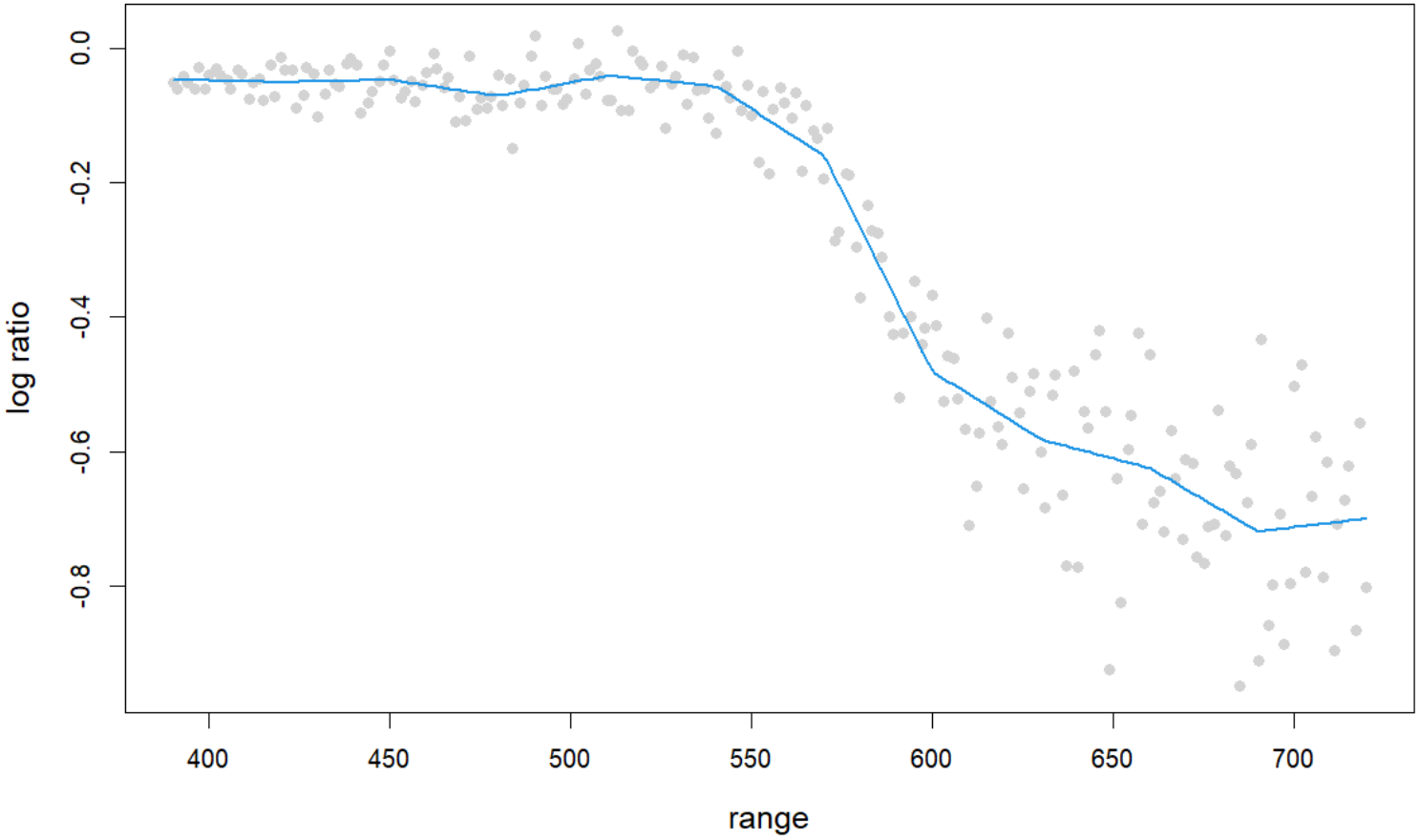
# Scatterplot smoothing

## Piecewise Linear basis – practical

```
## generate knots
> sj2 <- seq(min(lidar$range),max(lidar$range),length=12)
> X2 <- tf.X(lidar$range,sj2) ## get model matrix
> b2 <- lm(lidar$logratio ~ X2 - 1) ## fit model
> Xp2 <- tf.X(s,sj2) ## prediction matrix
## overlay estimated f
> lines(s,Xp2 %*% coef(b2), lwd=2, col=4)
```

# Scatterplot smoothing

## Piecewise Linear basis – practical



# Scatterplot smoothing

## Exercise 3

# Scatterplot smoothing

## Penalized regression

We fitted two models, both fit the data relatively well, but one is more wiggly than the other

The degree of model smoothness is controlled by the number of knots, $k$.

Choosing the degree of smoothing:

- backward model selection
- penalized regression

# Scatterplot smoothing

## Penalized regression

We fitted two models, both fit the data relatively well, but one is more wiggly than the other

The degree of model smoothness is controlled by the number of knots, $k$.

Choosing the degree of smoothing:

- ~~backward model selection~~
- penalized regression: choose $k$ a little larger than it is believed to be reasonably necessary

# Scatterplot smoothing

## Penalized Regression

In penalized regression, to find estimates for $\boldsymbol{\beta}$, we minimize:

$$\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2 + \lambda \boldsymbol{\beta}^T \mathbf{S} \boldsymbol{\beta}$$

where $\lambda$ is the smoothing parameter and $\boldsymbol{\beta}^T \mathbf{S} \boldsymbol{\beta}$ is a penalty.

The penalized least squares estimator of $\boldsymbol{\beta}$, is:

$$\widehat{\boldsymbol{\beta}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{S})^{-1} \mathbf{X}^T \mathbf{y}$$

# Scatterplot smoothing

## Penalized Regression

The penalty $\boldsymbol{\beta}^T \mathbf{S} \boldsymbol{\beta}$ shrinks the coefficients of the basis functions towards zero

The smoothing parameter $\lambda$ controls the extent of the shrinkage (and thus the smoothness):

$$\begin{aligned} \lambda = 0: &\quad \text{unpenalized } \boldsymbol{\beta} \Rightarrow \text{wiggly line} \\ \lambda \to \text{inf}: &\quad \boldsymbol{\beta} \approx 0 \Rightarrow \text{straight line} \end{aligned}$$

# Scatterplot smoothing

## Penalized Regression – practical

```
> prs.fit <- function(y,x,xj,sp) {
+  X <- tf.X(x,xj) ## model matrix
+  D <- diff(diag(length(xj)), differences=2) ## sqrt penal
+  X <- rbind(X,sqrt(sp)*D) ## augmented model matrix
+  y <- c(y,rep(0,nrow(D))) ## augmented data
+  lm(y ~ X - 1) ## penalized least squares fit
}
```

# Scatterplot smoothing

## Penalized Regression – practical

```
> lambda = 10
> b3 <- prs.fit(lidar$logratio, lidar$range, sj2, lambda)
> Xp3 <- tf.X(s,sj2) ## prediction matrix

> lambda = 100
> b4 <- prs.fit(lidar$logratio, lidar$range, sj2, lambda)
> Xp4 <- tf.X(s,sj2) ## prediction matrix

> lambda = 1000
> b5 <- prs.fit(lidar$logratio, lidar$range, sj2, lambda)
> Xp5 <- tf.X(s,sj2) ## prediction matrix
```

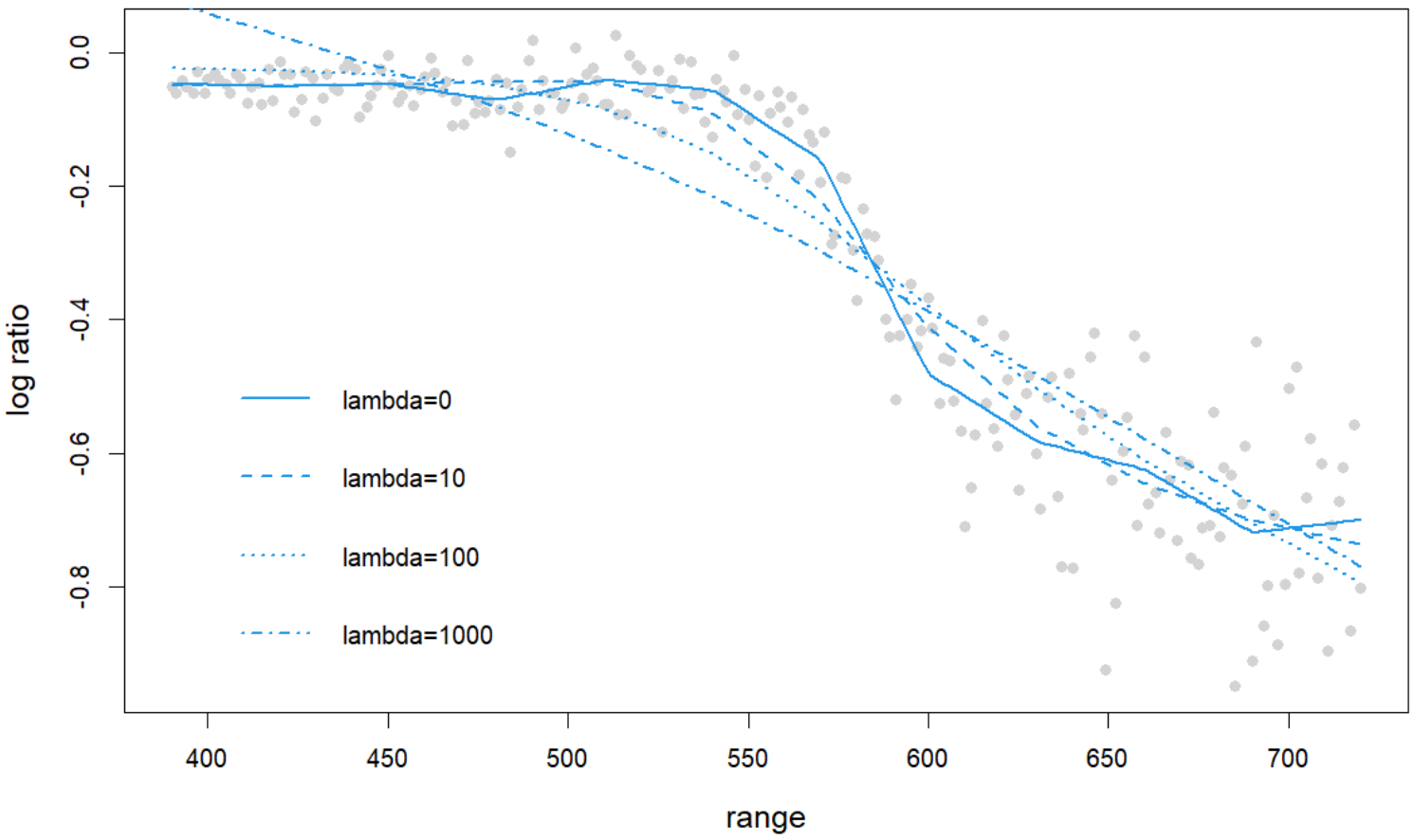
# Scatterplot smoothing

## Penalized Regression – practical

```
> plot(x=lidar$range,lidar$logratio,
+ xlab="range", ylab="log ratio",
+ cex.lab=1.25, pch=16, col="lightgray")
> lines(s,Xp2 %*% coef(b2), lwd=2, col=4)
> lines(s,Xp3 %*% coef(b3), col=4, lwd=2, lty=2)
> lines(s,Xp4 %*% coef(b4), col=4, lwd=2, lty=3)
> lines(s,Xp5 %*% coef(b5), col=4, lwd=2, lty=4)
> legend(x=400,y=-0.4, col=4, lty=1:4, lwd=2, bty="n",
+ legend=c("lambda=0","lambda=10","lambda=100",
+ "lambda=1000"))
```

# Scatterplot smoothing

## Penalized Regression



# Scatterplot smoothing

## Penalized Regression

Choose $\lambda$ so that $\hat{f}(x_i)$ is as close as possible to $f(x_i)$:

Use *cross-validation* criterion:

$$\mathrm{CV}(\lambda) = \sum_{i=1}^{n} (y_i - \hat{f}_{-i}(x_i; \lambda))^2$$

where $\hat{f}_{-i}$ the model fitted to all data except $y_i$

CV-based choice of $\lambda$ is the one that minimizes $\mathrm{CV}(\lambda)$ over $\lambda \geq 0$.

Computationally costly to calculate the *cross-validation* criterion

Efficient algorithms for computation of CV($\lambda$) have been proposed

Use *generalized cross-validation (GCV)* which is an approximation of the CV-criterion.

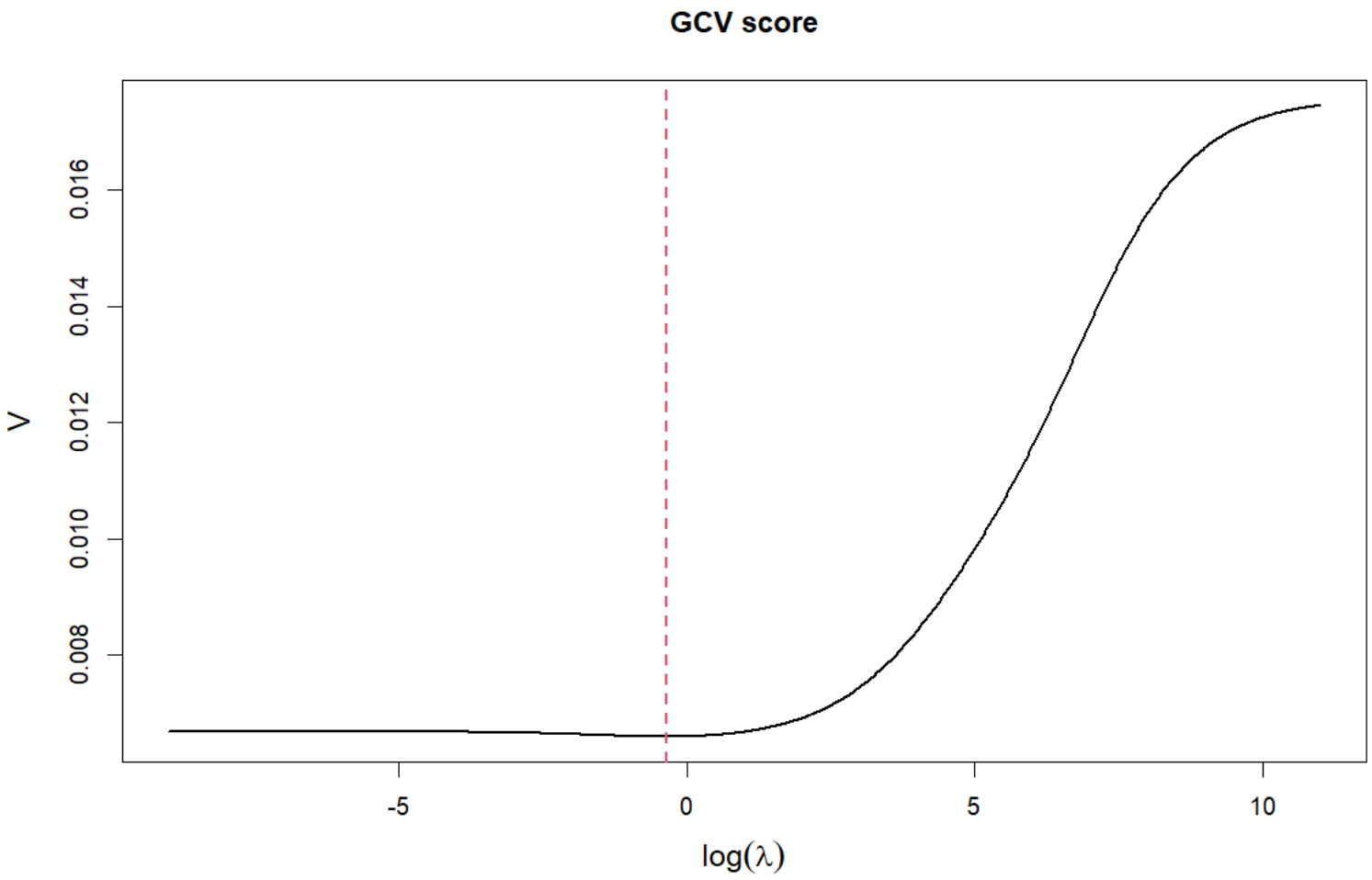
# Scatterplot smoothing

## Penalized Regression – practical

```
> loglambda = seq(-9,11,length=1000)
> n <- length(lidar$logratio)
> V <- rep(NA,1000)

> for (i in 1:1000) { ## loop through smoothing params
+  b <- prs.fit(lidar$logratio, lidar$range, sj2,
+  exp(loglambda[i])) ## fit model
+  trF <- sum(influence(b)$hat[1:n]) ## extract EDF
+  rss <- sum((lidar$logratio-fitted(b)[1:n])^2) ## residua
+  V[i] <- n*rss/(n-trF)^2 ## GCV score
}
> logsp <- loglambda[which.min(V)]
> plot(loglambda,V,type="l",xlab=expression(log(lambda)),
+ main="GCV score", cex.lab=1.25, lwd=2)
> abline(v=logsp,lty=2, col=2, lwd=2)
```

# Scatterplot smoothing

## Penalized Regression – practical



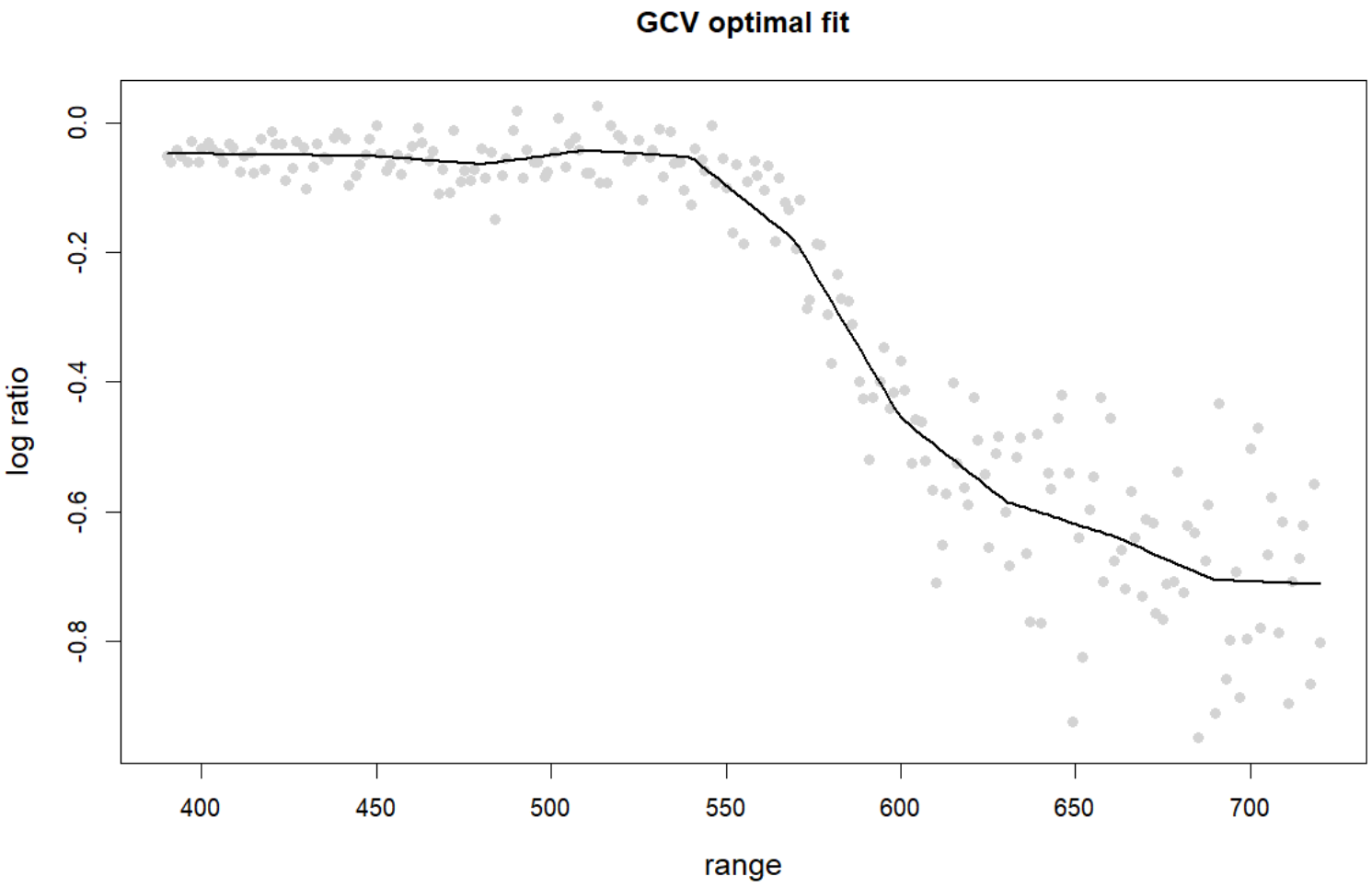
# Scatterplot smoothing

## Penalized Regression – practical

```
> b <- prs.fit(lidar$logratio, lidar$range,
+ sj2, exp(logsp)) ## re-fit

> plot(x=lidar$range,lidar$logratio,
+ xlab="range", ylab="log ratio",
+ cex.lab=1.25, pch=16, col="lightgray",
+ main="GCV optimal fit")
> lines(s, Xp2 %*% coef(b), col=1, lwd=2)
```

# Scatterplot smoothing

## Penalized Regression – practical

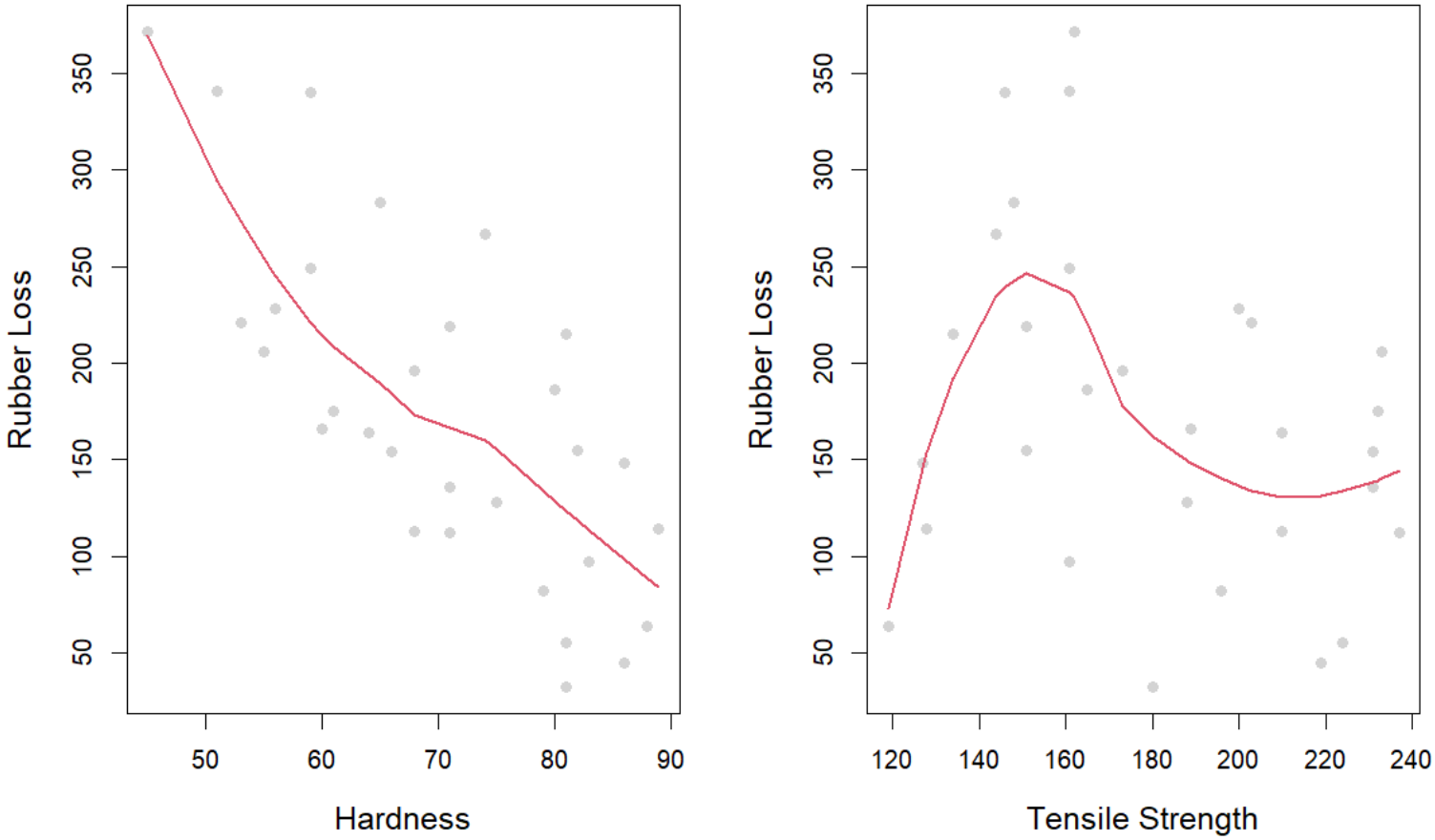


# Scatterplot smoothing

## Exercises 4 & 5

# Additive models

# Introduction




We would like to find the relation between rubber loss, rubber hardness and tensile strength

# Introduction

We showed how to construct flexible regression models for a single continuous predictor modeled as a smooth function (*scatterplot smoothing*)

However, there are two continuous covariates that may have nonlinear relationships with the response.

$\Rightarrow$ Additive models

# Additive models

## Introduction

An additive model is a linear model with a linear predictor involving a sum of smooth functions of covariates.

For the *Rubber* dataset following simple additive model could be fitted:

$$y_i = \alpha + f_1(x_{1i}) + f_2(x_{2i}) + \varepsilon_i$$

with $y_i$ rubber loss, $\alpha$ the intercept, $f_1$ and $f_2$ smooth functions for rubber hardness ($x_1$) and tensile strength ($x_2$), and $\varepsilon \sim \mathcal{N}(0, \sigma^2)$

# Additive models

## Introduction

Two remarks:

i. assumption of additive effects: $f_1(x_1) + f_2(x_2)$ is a quite restrictive special case of the general smooth function of two variables $f(x_1, x_2)$

ii. identifiability issue: any constant could be simultaneously added to $f_1(x_1)$ and subtracted from $f_2(x_2)$, without changing the model predictions.

# Additive models

## Basis functions

Use piecewise linear basis to represent the smooth functions:

$$f_1(x_1) = \sum_{j=1^{k_1}} b_j(x_1)\delta_j$$
$$f_2(x_2) = \sum_{j=1^{k_2}} q_j(x_1)\gamma_j$$

A penalty is also associated with each function:

$$\boldsymbol{\delta}^T \mathbf{S}_1 \boldsymbol{\delta} \text{ and } \boldsymbol{\gamma}^T \mathbf{S}_2 \boldsymbol{\gamma}$$

# Additive models

## Identifiability issue

We cannot estimate $\alpha$, $f_1(x_1)$ and $f_2(x_2)$ without a constraint.

Setting $\alpha$ to zero doesn't help.

Constrain $f_1(x_1)$ and $f_2(x_2)$ with a sum-to-zero constraint:

$$\sum_{i=1}^{n} f_1(x_1) = 0 \text{ or } \mathbf{1}^\mathsf{T}\mathbf{X}_1\boldsymbol{\delta} = 0$$
$$\sum_{i=1}^{n} f_2(x_2) = 0 \text{ or } \mathbf{1}^\mathsf{T}\mathbf{X}_2\boldsymbol{\gamma} = 0$$

$\Rightarrow$ use a column-centred-rank-reduced-basis which satisfies the identifiability constraint

# Additive models

## Model fitting

Setting

$$\begin{aligned} \boldsymbol{\beta}^T &= (\alpha, \boldsymbol{\delta}^T, \boldsymbol{\gamma}^T) \\ \mathbf{X} &= (\mathbf{1}, \mathbf{X}_1, \mathbf{X}_2) \end{aligned}$$

allows to reformulate additive models as follows:

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$$

# Additive models

## Model fitting

The estimates $\hat{\boldsymbol{\beta}}$ can be found by minimization of:

$$\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2 + \lambda_1 \boldsymbol{\beta}^T \mathbf{S_1} \boldsymbol{\beta} + \lambda_2 \boldsymbol{\beta}^T \mathbf{S_2} \boldsymbol{\beta}$$

with $\lambda_1$ and $\lambda_2$ the smoothing parameters for $f_1(x_1)$ and $f_2(x_2)$

Now that we have two smoothing parameters, grid searching for the GCV optimal values starts to become inefficient

$\Rightarrow$ use optimization approaches to find $\boldsymbol{\lambda}$.

```
> tf.XD <- function(x,xk,cmx=NULL,m=2) {
  ## get X and D subject to constraint
+  nk <- length(xk)
+   X <- tf.X(x,xk)[,-nk] ## basis matrix
+   D <- diff(diag(nk),differences=m)[,-nk] ##penalty
+   if (is.null(cmx)) cmx <- colMeans(X)
+   X <- sweep(X,2,cmx) ## subtract cmx from columns
+   list(X=X,D=D,cmx=cmx)
}
```

# Additive models

## Model fitting – practical

```
> am.fit <- function(y,x1,x2,sp,k=10) {
+  x1k <- seq(min(x),max(x1),length=k)
+  xdx1 <- tf.XD(x1,x1k)
+  x2k <- seq(min(x2),max(x2),length=k)
+  xdx2 <- tf.XD(x2,x2k)
+  nD <- nrow(xdx1$D)*2
+  sp <- sqrt(sp)
+  X <- cbind(c(rep(1,nrow(xdx1$X)),rep(0,nD)),
+             rbind(xdx1$X,sp[1]*xdx1$D,xdx2$D*0),
+             rbind(xdx2$X,xdx1$D*0,sp[2]*xdx2$D))
+  y1 <- c(y,rep(0,nD))
+  b <- lm(y1 ~ X - 1)   ## fit model
+  n <- length(y)
+  trA <- sum(influence(b)$hat[1:n]) ## EDF
+  rsd <- y - fitted(b)[1:n] ## residuals
+  rss <- sum(rsd^2) ## residual SS
+  sig.hat <- rss/(n-trA) ## residual variance
```

```
> am.gcv <- function(lsp,y,x1,x2,k) {
  ## function suitable for GCV optimization by optim
+    am.fit(y,x1,x2,exp(lsp),k)$gcv
}
```

# Additive models

## Model fitting – practical

```
> library(MASS)
> data(Rubber)
> nbKnots <- 15
> fit <- optim(c(0,0), am.gcv, y=Rubber$loss,
+  x1=Rubber$hard, x2=Rubber$tens, k=nbKnots)
> sp <- exp(fit$par)
> fit <- am.fit(Rubber$loss, Rubber$hard, Rubber$tens,
+  sp, k=nbKnots)
```

# Additive models

## Model fitting – practical

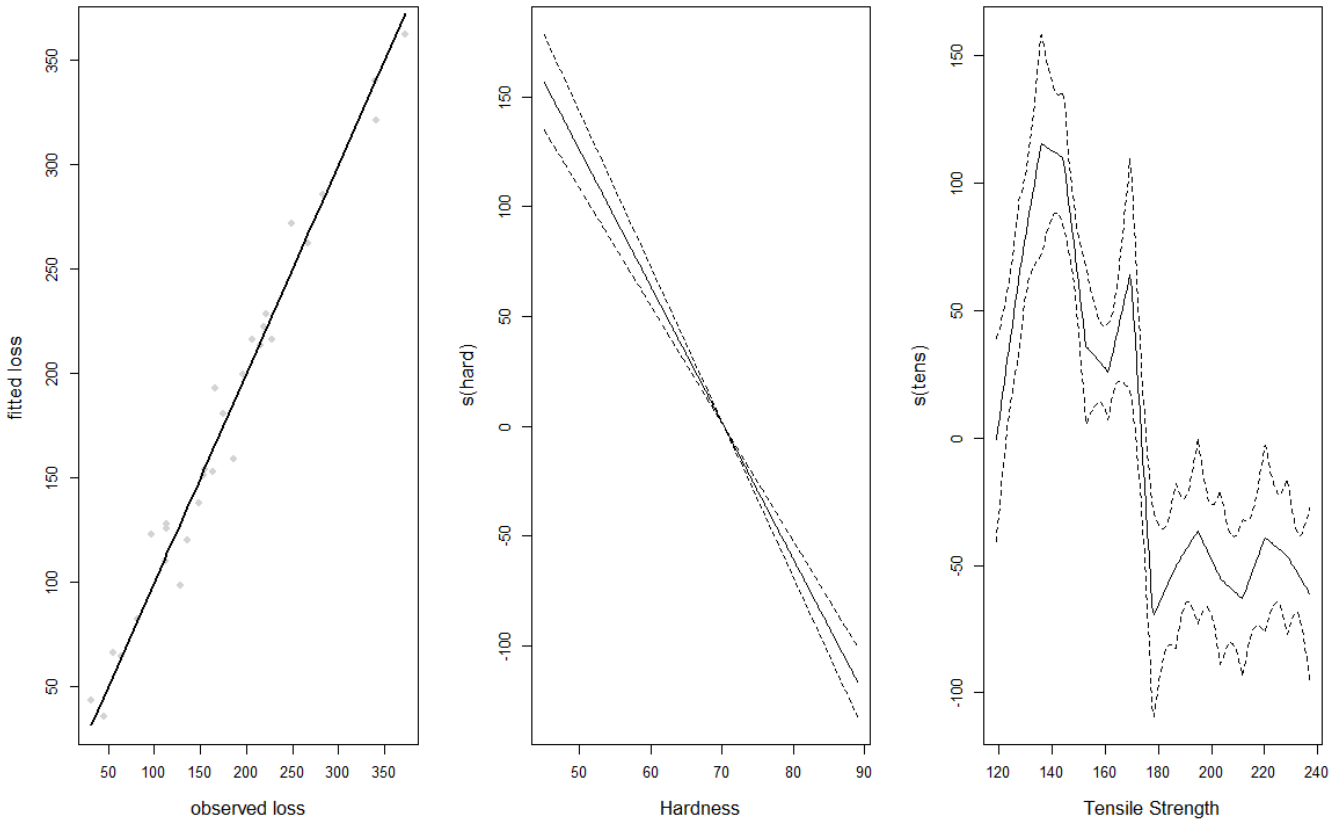



Figure: The best fit two term additive model ($k = 15$) for the Rubber data.

Notice that the smooth of rubber hardness (*hardness*) is estimated to be a straight line, and as a result its confidence interval has zero width at some point.

The zero width point in the interval occurs because the sum to zero constraint exactly determines where the straight line must pass through zero.

# Generalized additive models

Generalized additive model (GAM):

$$
\begin{aligned}
g(\mu_i) &= \alpha + \sum_l f_l(x_{l1}) \\
\mathrm{E}(y_i) &\equiv \mu_i
\end{aligned}
$$

Response $y_i$: distribution from family of exponential distributions: Binomial, Poisson, Gamma, Normal, ....

GAM will be fitted by penalized likelihood maximization (PIRLS)